

Synapse

Complete Project Guide & Viva Preparation

A Fork of Graphify - Knowledge-Graph Engine for Code

B.Tech Final Year Project - Phase 1

Prepared for Project Presentation, Technical Viva, and Team Reference

Compiled by tracing the actual source code, git history, and test suite of the project repository.
Every architectural claim, algorithm description, and measurement in this document is anchored to the codebase itself.

Table of Contents

<u>1. Executive Summary</u>	5
<u>2. What Is Graphify?</u>	6
<u>3. Original Graphify - Architecture & Pipeline</u>	7
3.1 The pipeline, accurately described	7
3.2 How a file becomes graph nodes and edges (concrete example)	7
3.3 Cross-file resolution - the hard part	8
3.4 Deduplication - three layers	8
3.5 Security model (original Graphify, unchanged by us)	9
3.6 The MCP server - a separate program, not a CLI subcommand	9
<u>4. Where the Fork Happened (Git Evidence)</u>	10
<u>5. Our Contributions - Overview</u>	11
<u>6. Original vs. Our Version - Comparison Table</u>	12
<u>7. Deep Dive: Our Four Backend Contributions</u>	14
7.1 Weighted Retrieval	14
7.2 Community-Aware Retrieval	14
7.3 Semantic Graph	15
7.4 Bidirectional Reasoning + Pre-Execution Validation	16
<u>8. Deep Dive: The Synapse VS Code Extension</u>	18
8.1 Backend bootstrap	18
8.2 UI surfaces	18
8.3 Notable implementation decisions	18
<u>9. Codebase Walkthrough</u>	20
9.1 Core pipeline (original Graphify)	20
9.2 Our backend contributions	21
9.3 The VS Code extension	21
9.4 Eval & tests	22
<u>10. End-to-End Execution Flow</u>	23
<u>11. AI / LLM Components</u>	24
11.1 Where LLMs are used, and where they are deliberately NOT used	24
11.2 Supported backends (multi-provider, not Claude-only)	24
11.3 Request/response shape (semantic extraction, representative of the pattern)	24

11.4 Prompt injection defense (a real, cited security control)	25
11.5 Cost, latency, rate limits (as actually measured)	25
<u>12. Graph Processing Explained</u>	26
<u>13. Technology Stack</u>	28
<u>14. Architecture Diagrams</u>	29
Diagram 1 - Overall System Architecture (Synapse today)	29
Diagram 2 - Original Graphify Pipeline	29
Diagram 3 - Query-Time Architecture (where our features plug in)	30
Diagram 4 - End-to-End Request Flow (Copilot Chat -> Synapse -> back)	30
Diagram 5 - LLM / API Interaction (semantic extraction & reasoning, shared pattern)	31
Diagram 6 - Our Feature Interconnections	31
<u>15. Important Code Snippets</u>	32
<u>16. Dependencies & Environment / How to Run</u>	34
16.1 Core dependencies (pyproject.toml)	34
16.2 VS Code extension dependencies	34
16.3 Environment variables (selected, all read directly via os.environ/os.getenv, no .env.example exists in the repo)	34
16.4 How to run the project end-to-end	34
<u>17. Accuracy Evaluation - What We Measured</u>	36
17.1 Retrieval accuracy (graph traversal only, no LLM)	36
17.2 CrossCodeEval - real LLM code completion (the metric that matters most)	36
17.3 Regression testing	37
<u>18. Limitations & Future Work</u>	38
18.1 Currently implemented (do not overclaim these as future work)	38
18.2 Known limitations (from the actual implementation, not guessed)	38
18.3 Possible future work (explicitly NOT implemented yet - from the team's own research docs)	38
<u>19. How to Explain Our Contributions During the Presentation</u>	40
Contribution 1 - Weighted Retrieval	40
Contribution 2 - Community-Aware Retrieval	40
Contribution 3 - Semantic Graph	40
Contribution 4 - Bidirectional Reasoning + Pre-Execution Validation	41
Contribution 5 - Synapse VS Code Extension	41
<u>20. Viva Preparation - Full Q&A Bank</u>	43
Basic Questions	43

Technical Questions	43
Architecture Questions	44
AI/LLM Questions	44
Algorithm Questions	45
Code Questions	45
"Why did you use X?" Questions	45
"What happens if X fails?" Questions	46
Contribution Questions	46
Difficult / Tricky Questions	46
<u>21. Learning Path: New to the Project</u>	48
<u>22. Cheat Sheet</u>	50
Project in 30 seconds	50
Project in 1 minute	50
Project in 3 minutes	50
Original Graphify	50
Our contributions	50
Architecture	50
Technologies	50
Most important algorithms	51
Most important files	51
Most important functions	51
20 must-know viva questions	51
<u>23. Things the Team Must Verify</u>	53
<u>24. Appendix - File & Function Index</u>	54

1. Executive Summary

Synapse is a fork of an open-source tool called **Graphify** - a Python CLI that turns a codebase (plus its docs, papers, images, and video) into a queryable **knowledge graph**: nodes are code/concept entities, edges are relationships (`calls`, `imports`, `inherits`, ...), and every edge carries a confidence label (`EXTRACTED`, `INFERRED`, `AMBIGUOUS`). The graph is meant to give an LLM (or a developer) compact, structurally-grounded context about a codebase instead of dumping raw files at it.

Our team took this engine and added four backend features plus a full VS Code extension:

1. **Weighted retrieval** - graph traversal that prefers strong relationships (`calls`) over weak ones (`imports`) instead of treating every edge as an equal hop.
2. **Community-aware retrieval** - restricts a query to the relevant cluster of the codebase first, so retrieval doesn't wander across unrelated modules.
3. **Semantic graph** - an opt-in LLM pass that adds *intent* edges (`handles`, `validates`, `manages`, ...) on top of the structural AST graph.
4. **Bidirectional reasoning + pre-execution validation** - an LLM answers a question with typed, checkable claims; those claims are verified against the graph; contradictions trigger a self-correction pass; and a separate hook can block an AI coding agent from writing code that contradicts the graph *before* it executes.
5. **Synapse VS Code extension** - a from-scratch TypeScript extension (Dashboard, sidebar, graph visualization, onboarding, and two GitHub Copilot Chat tools) that wraps the Python CLI.

We measured the effect of these changes with two accuracy harnesses (a graph-retrieval precision/recall test and a real-LLM CrossCodeEval code-completion benchmark). Headline result: our retrieval additions improved real code-completion exact-match accuracy from **21.1% (existing Graphify) to 31.6% (Synapse)** on a paired 19-task sample, and from **25.5% -> 29.8%** on separate fresh runs - while being honest that on a narrow single-repo precision@10 metric, plain keyword grep still edges out both graph-based approaches (see Section 17).

2. What Is Graphify?

The problem it solves: when an LLM (or a developer) needs to understand a codebase, the usual options are "dump entire files into context" (expensive, hits context limits, indiscriminate) or "grep for keywords" (fast but structurally blind - it can't tell you *what calls what*). Graphify's answer: parse the codebase once into a graph, then answer future questions by walking the graph instead of re-reading files.

In one sentence: Graphify is a Claude Code skill backed by a standalone Python library/CLI that extracts structural and semantic relationships from a corpus of files and stores them as a NetworkX graph, which can then be queried, visualized, exported, or served over MCP.

Inputs: any mix of source code (~25 languages via tree-sitter), Markdown/docs, PDFs/papers, images, and audio/video.

Outputs (written to `graphify-out/`):

- `graph.json` - the graph itself, NetworkX node-link JSON format
- `GRAPH_REPORT.md` - a human-readable summary (god nodes, communities, surprises, suggested questions)
- `graph.html` / `graph.svg` - interactive/static visualizations
- An Obsidian-compatible vault (one Markdown note per node)
- Optional exports to Neo4j, FalkorDB, GraphML, Cypher

Confidence tagging (used throughout the whole system, original and ours alike):

Label	Meaning
EXTRACTED	Directly observed in source (an actual import statement, a real function call)
INFERRED	A reasonable deduction (cross-file call-graph matching, LLM-inferred relation) - carries a numeric <code>confidence_score</code>
AMBIGUOUS	Uncertain, flagged in the report for human review

3. Original Graphify - Architecture & Pipeline

3.1 The pipeline, accurately described

The project's own `ARCHITECTURE.md` describes the pipeline as a clean chain of seven functions: `detect()` -> `extract()` -> `build_graph()` -> `cluster()` -> `analyze()` -> `report()` -> `export()`. **This is a simplified narrative, not literal code** - our research confirmed that `build_graph()`, `analyze()`, and a single top-level `export()` do not exist as named functions anywhere in the codebase. The actual sequence of operations is real, but it lives as roughly **1,000 lines of imperative orchestration inside `graphify/cli.py`'s `extract` command handler** (and is partially duplicated, with variations, in `graphify/watch.py`'s `_rebuild_code()` for incremental updates). This is a good thing to know before a panel asks you to "show us the `build_graph` function" - point them at `cli.py`'s `extract` command instead, and be ready to explain why (see Section 23, item 1).

The seven conceptual stages, and what each *actually* calls:

Stage	What really happens	Key file(s)
Detect	<code>detect.detect(root)</code> walks the filesystem, classifies each file (<code>code/document/paper/image/video</code>), applies <code>.gitignore/.graphifyignore</code> , and filters out anything that looks like a secret/credential file	<code>graphify/detect.py</code>
Extract	<code>extract.extract(paths)</code> dispatches each file to a language-specific tree-sitter extractor, then runs cross-file resolution (import resolution, an INFERRED call-graph second pass, cross-language phantom-edge guards)	<code>graphify/extract.py</code> , <code>graphify/extractors/</code>
Build	<code>build.build_from_json(extraction)</code> assembles a NetworkX graph: deduplicates nodes/edges, reconciles AST-origin vs. LLM-origin node IDs, applies the language-family edge guard	<code>graphify/build.py</code>
Cluster	<code>cluster.cluster(G)</code> partitions the graph into communities using the Leiden algorithm (graspologic), falling back to Louvain if unavailable	<code>graphify/cluster.py</code>
Analyze	Five independent functions - <code>god_nodes()</code> , <code>surprising_connections()</code> , <code>suggest_questions()</code> , <code>find_import_cycles()</code> , <code>graph_diff()</code> - derive insight from the graph	<code>graphify/analyze.py</code>
Report	<code>report.generate()</code> (not <code>render_report()</code>) renders <code>GRAPH_REPORT.md</code> from all of the above	<code>graphify/report.py</code>
Export	<code>export.to_json/to_html/to_obsidian/to_svg/...</code> write the various output formats	<code>graphify/export.py</code> , <code>graphify/exporters/</code>

3.2 How a file becomes graph nodes and edges (concrete example)

Every extractor returns the same schema, enforced by `graphify/validate.py`:

```
{
  "nodes": [{"id": "unique_string", "label": "human name", "file_type": "code",
    "source_file": "path", "source_location": "L42"}],
```

```
"edges": [{"source": "id_a", "target": "id_b", "relation": "calls",
            "confidence": "EXTRACTED", "source_file": "path"}]
}
```

Extraction is **imperative AST walking, not tree-sitter's declarative query language** - every language extractor (25+ of them) recursively visits parse-tree nodes and dispatches on node type (`function_item`, `class_declaration`, `import_statement`, ...). We traced `graphify/extractors/rust.py` end to end as a representative example (chosen for being self-contained, no shared-engine indirection):

- `struct_item/enum_item/trait_item` -> a node per type; struct fields become `references` edges to their field type.
- `impl_item` -> resolves `impl Type` and, if present, `impl Trait for Type`, emitting `implements`; nested functions become `method` edges from the type to the function.
- `use_declaration` -> an `imports_from` edge (Rust imports don't resolve to a specific file, so the target is a bare-name node).
- Calls are resolved to a `calls` edge if the callee exists in the same file; otherwise deferred to a corpus-wide second pass in `extract.py`, unless the callee name is in a blacklist of generic trait methods (`new`, `clone`, `unwrap`, ...) kept out to avoid flooding the graph with meaningless high-degree "god nodes."

Most languages (Python, JS/TS, Java, C/C++, C#, Kotlin, Scala, PHP, Swift, Lua, ...) instead share one large engine, `graphify/extractors/engine.py` (~5,000 lines - the single largest file in the package), parameterized per-language by `LanguageConfig` objects. This file is dense with per-language special cases (C# interface-naming heuristics, Java lambda binding tables, Swift extension merging) accumulated across many real bugfixes - not something to memorize line-by-line, but worth knowing it exists and why it's large if asked "why is this file 5,000 lines."

3.3 Cross-file resolution - the hard part

A single file's AST only tells you what's declared *in that file*. The genuinely hard engineering problem is reconciling that across an entire multi-language, multi-file corpus. `extract.py`'s multi-file entry point does, in order:

1. Concatenates every file's raw extraction.
2. Runs a multi-stage **node-ID canonicalization** pass so an AST-origin node and an LLM-origin node describing the same file converge on one ID (`{parent-relative-path}_{entity-name}`).
3. Runs per-language cross-file import resolvers (Python absolute imports, Java, C#, Bash `source`).
4. Runs the **INFERRED call-graph second pass**: every extractor's unresolved bare-name calls (`raw_calls`) are matched against a corpus-wide `label -> node_ids` index, guarded by a **cross-language family check** so a Python `render()` call can't accidentally resolve to a JavaScript file's `render()`. A resolved call gets `EXTRACTED` confidence only if there's corroborating import evidence; otherwise it's `INFERRED` (0.8 confidence).

3.4 Deduplication - three layers

1. **Within a file**: each extractor tracks a `seen_ids` set - a duplicate definition in the same file is collapsed.
2. **Between files (exact ID match)**: NetworkX's `add_node()` is idempotent; adding the same ID twice just overwrites attributes (semantic nodes intentionally added after AST nodes so they win, since they carry richer cross-file context).

3. **Fuzzy entity dedup** (`graphify/dedup.py`) for non-code nodes only (docs/concepts - code identity is the fully-qualified ID, never merged by label alone): a pipeline of exact normalization -> Shannon-entropy gate (rejects short/generic labels) -> **MinHash + LSH blocking** (to avoid $O(n^2)$ all-pairs comparison) -> **Jaro-Winkler similarity verification** (threshold 92.0) -> several false-positive guards (numeric-suffix siblings, cross-file document boilerplate) -> union-find merge.

Worth knowing: `graphify/_minhash.py` is a **from-scratch reimplementaion** of the standard `datasketch` MinHash/LSH library. Reason (per its own module docstring): `datasketch` transitively imports `scipy.integrate.quad`, which on Windows can trigger a `platform.machine()` call that spawns `cmd.exe` and hangs for minutes under certain corporate security software. This is a real, cited engineering workaround - a good example of a subtle platform-compatibility bug fixed by reimplementing rather than patching a dependency.

3.5 Security model (original Graphify, unchanged by us)

All external input passes through `graphify/security.py` before use:

Input type	Guard
URLs (for ingest)	<code>validate_url()</code> - http/https only, blocks cloud-metadata hosts, resolves DNS and rejects private/loopback/link-local ranges, re-validates on every redirect (defeats DNS-rebinding SSRF)
Fetched content	<code>safe_fetch()/safe_fetch_text()</code> - streamed with a hard byte cap, aborts mid-download rather than materializing an oversized body
Graph file paths (MCP server)	<code>validate_graph_path()</code> - must resolve inside a <code>graphify-out/-</code> named directory that already exists
Node labels	<code>sanitize_label()</code> - strips control characters, caps length; HTML output additionally escapes
LLM prompts	Every file's content is wrapped in <code><untrusted_source></code> tags with a SHA-256 fingerprint, and known jailbreak/chat-template tokens (<code>< im_start ></code> , <code>[INST]</code> , forged closing tags) are defanged with zero-width-space insertion before the text ever reaches the model

3.6 The MCP server - a separate program, not a CLI subcommand

Important, easy-to-miss detail: `graphify serve` **is not a subcommand of the `graphify` CLI**. It is a completely separate console-script entry point declared in `pyproject.toml` (`graphify-mcp = "graphify.serve:_main"`), with its own `argparse` parser (`--transport stdio|http`, `--host`, `--port`, `--api-key`). It exposes MCP tools like `query_graph`, `get_node`, `get_neighbors`, `god_nodes`, `shortest_path`, `list_prs` - and, as of this project's changes, also our `reason_with_graph` and `validate_proposal` tools (see Section 7.4).

4. Where the Fork Happened (Git Evidence)

This section is based on `git log`, `git diff --stat`, and reading actual diffs - not assumption.

```
f5a3592 chore(release): 0.9.27          <- last real upstream Graphify commit
a6737ac chore: clean Graphify baseline codebase  <- reformatting pass (see below)
5db7e4e chore: remove upstream files            <- removes upstream-only assets
e321d3d Update README.md                      <- our team's work begins here
... docs: add local setup guide
... eval: add retrieval accuracy + CrossCodeEval accuracy metrics
b5a6299 feat: add v1 VS Code extension (graphify-vscode)
... VS Code extension fixes, onboarding, rebranding to "Synapse"
fdb2bd2 feat: weighted retrieval              +
67b21f2 feat: add community retrieval          | three feature branches,
9c9d560 feat: add semantic graph                + by contributor Charuvarthan-T
9836948 feat: add Synapse reasoning and preexec validation
81bdc08 (this session) merge origin/main to reunify the above with local main
```

Important nuance: `a6737ac` ("clean Graphify baseline codebase") looks at first glance like it made huge changes - it touches nearly every file in `graphify/`, with diffs in the hundreds of lines per file. We verified directly (`git diff a6737ac^ a6737ac -- graphify/build.py`) that this is **not a functional rewrite** - it strips detailed explanatory comments (e.g. a 20-line comment block explaining the three-layer dedup strategy) and reformats code (e.g. converting compact one-line dict literals into one-key-per-line). The underlying logic is unchanged. `5db7e4e` then deletes genuinely upstream-only assets: translated READMEs (18 languages), logo/branding SVGs, a GitHub Actions publish workflow, a benchmarks doc, and worked example outputs.

What this means for the "original vs. ours" line: everything in `graphify/` as of commit `5db7e4e` is functionally the real upstream Graphify engine (just de-commented and reformatted). Our team's actual code contributions begin at `e321d3d` and after.

A divergence we found and fixed during this analysis: the local `main` branch and GitHub's `origin/main` had diverged - both had the three retrieval features, but only `origin/main` had the reasoning/preexec-validation feature (commit `9836948`, ~1,400 lines: `bidirectional_reasoner.py`, `claims.py`, `graph_checks.py`, `preexec_validate.py`, two research docs, two test files). We merged `origin/main` into local `main` (clean merge, no conflicts - confirmed the diff between the two branches was an exact superset) so this document reflects the complete, current project. **The team should push this merge** so GitHub and local history agree going forward (see Section 23).

5. Our Contributions - Overview

#	Contribution	Files (new)	Lines added	Opt-in / default
1	Weighted retrieval	<code>weights.py</code> , <code>weighted_retrieval.py</code>	~720	Default-on (<code>--unweighted</code> to disable)
2	Community-aware retrieval	<code>community_detection.py</code> , <code>community_index.py</code> , <code>community_retrieval.py</code>	~700	Opt-in (<code>--community-aware</code>)
3	Semantic graph	<code>semantic_extraction.py</code> , <code>semantic_relations.py</code> , <code>semantic_graph.py</code>	~500	Opt-in, separate command (<code>graphify semantic-graph</code>)
4	Bidirectional reasoning + preexec validation	<code>bidirectional_reasoner.py</code> , <code>claims.py</code> , <code>graph_checks.py</code> , <code>preexec_validate.py</code>	~1,400	<code>reason</code> command: revision on by default. <code>preexec-check</code> : opt-in via env var/flag
5	Synapse VS Code extension	entire <code>vscode-extension/</code>	~2,000 (TS)	N/A - a new client application
6	Eval tooling	<code>tools/eval/*.py</code> , <code>docs/eval/*.md</code>	~800	N/A - internal measurement tooling

All four backend features (1-4) were built by the same contributor (Charuvarthan-T) and each ships with its own research/rationale doc under `docs/research/` and its own dedicated test file - read those docs; they are primary-source "why we built it this way" material, quoted throughout Section 7.

6. Original vs. Our Version - Comparison Table

Aspect	Original Graphify	Our Version (Synapse)	Why We Changed It
Structural code graph (AST, no LLM)	Yes tree-sitter, 25+ languages	Yes identical extractor, unchanged	N/A - kept as-is, it works
Graph traversal for queries	Unweighted BFS/DFS - every edge is one hop	Weighted BFS/DFS by default - strong relations (<code>calls</code>) preferred over weak ones (<code>imports</code>)	Legacy traversal couldn't distinguish "this function calls that one" from "this file happens to import that one"
Search scope	Always the whole graph	Optional community-aware restriction to the relevant cluster first	On larger repos, a few weak bridge edges can pull retrieval across unrelated modules
Relationship types	Structural only (<code>calls</code> , <code>imports</code> , <code>inherits</code> , ...) + a legacy multi-modal <code>semantically_similar_to</code> for docs/papers/images	+ a closed vocabulary of code-intent edges (<code>handles</code> , <code>validates</code> , <code>manages</code> , <code>responsible_for</code> , <code>related_to</code>) via an opt-in LLM pass	AST edges say <i>what</i> connects to <i>what</i> , never <i>why</i>
Answer verification	None - an LLM reading graph context could still hallucinate	Bidirectional reasoning : LLM claims are extracted, checked against the graph, and revised if contradicted; refuses to present an uncorrected contradiction as fact	GraphRAG context reduces hallucination but doesn't eliminate it; this adds an explicit verification step
Guarding AI-agent code edits	None	Pre-execution validation hook: can block (in strict mode) a proposed code write that calls a method the graph shows doesn't exist on that class	Stops one class of confidently-wrong AI-generated code before it's written, without blocking on merely-unknown symbols
Editor integration	None (CLI + Claude Code skill only)	Full VS Code extension: Dashboard, sidebar, graph visualization, onboarding walkthrough, 2 GitHub Copilot Chat tools	Original tool was CLI-only; a visual, in-editor experience lowers the barrier to using it
MCP tool surface	<code>query_graph</code> , <code>get_node</code> , <code>get_neighbors</code> , <code>get_nodes</code> , <code>shortest_path</code> , <code>list_prs</code> , ...	+ <code>reason_with_graph</code> , <code>validate_proposal</code> ; existing <code>query_graph/shortest_path</code> retrofitted to use weighted/community-aware retrieval	Expose the new features to any MCP client, not just our own CLI/extension

Accuracy measurement	None in-repo	Two eval harnesses (<code>retrieval_accuracy.py</code> , <code>crosscodeeval.py</code>) + two written reports	Needed real numbers, not intuition, to know whether the new retrieval features actually help
API key requirement to build a graph	No (AST-only extraction is free)	Still no - weighted/community-aware retrieval are pure graph algorithms; only the semantic-graph and reasoning features are opt-in and need an LLM key	Preserving the "no API key needed for the core graph" guarantee was an explicit design constraint

7. Deep Dive: Our Four Backend Contributions

7.1 Weighted Retrieval

Problem: legacy Graphify's traversal treats every edge as an equal-cost hop. A question like "how does X execute" benefits from following `calls` edges preferentially over incidental `imports` edges, but unweighted BFS/DFS has no way to express that.

How it works:

- `graphify/weights.py`'s `RelationWeightRegistry` holds a curated importance table (`calls=1.0 > inherits=0.9 > implements=0.85 > references=0.6 > imports=0.4 > semantically_similar_to=0.2`, extendable via `.register()` with no call-site edits needed elsewhere), further softened by confidence tier (`EXTRACTED=1.0x`, `INFERRED=0.85x`, `AMBIGUOUS=0.7x`).
- `graphify/weighted_retrieval.py` uses this table two different ways:
 - `find_shortest_path` converts importance to cost (`cost = 1/importance`) and calls `networkx.shortest_path(graph, weight="weight")` - this is genuinely Dijkstra's algorithm (NetworkX dispatches to Dijkstra for weighted shortest-path).
 - `expand_neighborhood` (used for open-ended queries, not point-to-point paths) is **not** Dijkstra - it's a **hop-bounded frontier expansion where each hop's candidates are sorted by descending edge importance** before being added to the next frontier. This deliberately keeps the same hop-count semantics as the old unweighted BFS/DFS (same API, same depth limit) while changing *which* neighbors are prioritized within that depth.
- **The clearest before/after example** (from `tests/test_weighted_retrieval.py`): given a direct 1-hop `imports` edge `A->D` and a 2-hop `calls` chain `A->B->D`, **unweighted picks the 1-hop path; weighted picks the 2-hop path** - because two `calls` edges cost $2 \times (1/1.0) = 2$, while one `imports` edge costs $1/0.4 = 2.5$. Weighted retrieval is willing to take *more hops* for *stronger relationships*.
- Wired in as **default-on** for `graphify query` and `graphify path` (`--unweighted` opts out) and exposed as a `weighted: bool` parameter (default `true`) on the MCP `query_graph/shortest_path` tools.

Named future work (from the team's own `docs/research/weighted-graph-retrieval.md`): learning weights from query-log feedback instead of a fixed table, query-type-conditioned weight profiles, and cost-bounded (not just hop-bounded) traversal.

7.2 Community-Aware Retrieval

Problem: even with weighted edges, retrieval still searches the *entire* graph. On a large codebase, a handful of legitimate-but-weak cross-module edges can pull a query's results into an unrelated part of the system.

How it works - and why it's a genuinely separate module, not a rename of the original clustering:

- `graphify/cluster.py` (original) is a single hardcoded function used **once, at build time**, to stamp a `community` attribute onto the saved graph.
- `graphify/community_detection.py` (new) defines a **pluggable strategy interface** (`CommunityDetector ABC`) with four interchangeable algorithms - Louvain, Leiden, Label Propagation (a genuinely new algorithm not present in `cluster.py` at all), and an `Auto` selector - used at **query time**. Its `auto` mode deliberately mirrors `cluster.py`'s Leiden-then-Louvain preference for consistency, but it independently

handles degenerate cases (empty graphs, graphs with no edges, fewer than 3 nodes) that only matter for on-the-fly query-time detection.

- `graphify/community_index.py` builds a lightweight lookup (`CommunityIndex`) that **prefers the community labels already stored in `graph.json`** from build time, and only runs live detection as a fallback - an explicit performance decision.
- `graphify/community_retrieval.py` scores which communities are relevant to a query's seed nodes and search terms, computes a **confidence score** (`gap_confidence x seed_coverage`), and only restricts the search if confidence clears a threshold (default 0.55). If seeds span multiple communities, or confidence is too low, it **falls back to the full graph** rather than guessing - a safety-first design. When it does restrict, it hands the smaller subgraph straight to the existing `expand_neighborhood()` (weighted retrieval) - communities never reimplement traversal, they only narrow what's searched.
- Wired in as **opt-in** (`--community-aware`) for `graphify query`, composable with `--weighted/--unweighted` since the two flags are independent. The MCP server surfaces a `community_aware: bool` (default `false`) parameter, and prints a transparency header (`Communities=[...] (confidence=0.82)` or `Community fallback (low_confidence)`) so results are auditable, not a black box.

Named future work: persisting the community index alongside `graph.json` for cold-start speed, multi-resolution (parent/child) communities, and replacing the hard subgraph restriction with a soft cost penalty.

7.3 Semantic Graph

Problem: structural (AST) edges tell you *what* connects to *what* - `LoginHandler --calls--> TokenValidator` - but never *why*. That "why" (intent, responsibility) previously had no place in the graph for code, because Graphify's LLM-based semantic extraction was deliberately restricted to docs/papers/images (code is already covered structurally).

How it works - traced end to end:

1. `graphify semantic-graph` (a separate, opt-in CLI command - **not** run automatically during `extract`) loads an existing `graph.json`.
2. `semantic_extraction.select_code_units()` filters to code nodes only, batches them (default 20 per batch), and builds a deterministic prompt listing each unit's ID/label/location, plus a **fixed closed vocabulary**: `{handles, validates, manages, responsible_for, related_to}`. The vocabulary is deliberately closed (not "invent any relation you like") to keep ranking and prompts deterministic.
3. The LLM call reuses the existing multi-backend `graphify.llm._call_llm()` - no new backend/API-key plumbing was written - wrapped in a bounded-retry provider.
4. `semantic_relations.parse_semantic_relations()` validates every returned relation (both endpoints resolvable, relation type in the closed vocabulary, no self-loops); unmatched endpoints become new `concept` nodes rather than being dropped.
5. **The one genuinely tricky design decision:** `semantic_graph.apply_semantic_relations()` merges results into the live graph. A plain graph edge is keyed only by `(source, target)` - if a structural edge already exists between two nodes, blindly calling `add_edge()` again would silently overwrite its `relation` field. So for a pair that **already has an edge**, the new semantic relation is appended to a `secondary_relations` list on that edge instead of replacing anything; only genuinely new pairs get a brand-new graph edge. Re-running the pass twice is idempotent (verified by test).
6. The five new relation types are registered in the same `RelationWeightRegistry` used by weighted retrieval (`handles=0.65, manages=0.65, responsible_for=0.6, validates=0.55, related_to=0.3`) - so a

structurally-weak edge that also carries a `handles` annotation ranks higher during retrieval, and semantic-only edges (between two previously-unconnected nodes) are directly traversable by both weighted and community-aware retrieval with no extra code, verified by dedicated integration tests.

Named future work: feeding actual source-code snippets (not just labels) to the LLM for richer context, per-file caching of extraction results, and a probed-but-unused `--multigraph` mode that would let structural and semantic edges become true parallel edges instead of a secondary-relation list - an explicitly acknowledged "known workaround, real fix pending."

7.4 Bidirectional Reasoning + Pre-Execution Validation

These are two related contributions sharing one foundation module. Team's own internal labels: **C2 = bidirectional reasoning**, **C3 = pre-execution ("preexec") validation**.

The shared foundation - `graphify/graph_checks.py`: a library of structural yes/no lookups whose cardinal rule is *absence of evidence is never treated as evidence of absence*. Every check returns one of three verdicts:

Verdict	Meaning
SUPPORTED	The graph directly shows this relationship exists
CONTRADICTED	The graph shows evidence that actively conflicts with the claim (e.g., the source node has an explicit <code>inherits</code> edge to a <i>different</i> class than claimed)
UNKNOWN	The graph doesn't mention this either way - not treated as false, because the graph may simply be incomplete

`check_exists()` can only return `SUPPORTED` or `UNKNOWN`, never `CONTRADICTED` - not finding something is not proof it doesn't exist. `check_relation()/check_method_of()` only escalate to `CONTRADICTED` when the graph *demonstrably enumerates* the relevant alternatives (e.g., a class's full known member list) and the claim isn't among them. Aggregating multiple checks is deliberately conservative: **any contradiction makes the whole answer `CONTRADICTED`; a mix of `SUPPORTED` and `UNKNOWN` (no contradictions) still rolls up to `UNKNOWN` overall**, "for safety" (the code's own comment) - the system refuses to call an answer fully validated if even one claim couldn't be checked.

Bidirectional reasoning (`graphify/bidirectional_reasoner.py`): "bidirectional" describes the *loop direction*, not a graph-search algorithm. Ordinary GraphRAG is one-directional: graph -> context -> LLM -> answer. This adds the reverse leg:

```
question -> retrieve graph context (reuses weighted + community-aware retrieval)
-> LLM answers AND emits typed claims (e.g. {"type":"inherits","source":"LoginHandler","target":"SessionStore"})
-> every claim checked against the graph (graph_checks)
-> if any claim is CONTRADICTED or UNKNOWN -> send the validation report back to the LLM
-> LLM revises its answer
-> re-validate the revision
-> if STILL contradicted -> refuse to present the draft as fact; show a cautionary message instead
```

Directly exercised by a test where a scripted LLM first (wrongly) claims `LoginHandler inherits SessionStore`; the graph shows it actually inherits `BaseHandler`; the contradiction triggers a revision; the corrected answer is accepted. If no LLM backend is configured at all, the function still returns the retrieved graph context with an honest "cannot reason without a provider" message rather than failing.

Pre-execution validation (`graphify/preexec_validate.py`): validates an **AI coding agent's proposed**

code against the graph *before* it's written/executed. Flow: a proposed `Write/Edit` tool call is normalized into a `ProposedAction`; Python code is parsed with `ast` (falling back to regex for incomplete snippets) to extract imports, method calls, and attribute accesses; each reference is checked via `graph_checks`; results aggregate into a report. **The blocking rule is deliberately narrow**: a proposal is blocked only if the verdict is `CONTRADICTED` **and** strict mode is on. A merely `UNKNOWN` symbol (not blockable, ever, even in strict mode) - because a symbol simply not being in the graph is common and not evidence of a mistake. This is wired into Claude Code as an opt-in **PreToolUse hook** (`graphify hook-guard write --preexec`, gated by the `GRAPHIFY_PREEEXEC` environment variable, off by default so it never changes behavior for someone who hasn't opted in) and into the MCP server as a `validate_proposal` tool.

Concrete test example: a graph knows `PaymentManager` has methods `charge/refund`. Proposing code that calls `PaymentManager.missing_method()` -> `CONTRADICTED` -> blocked in strict mode. Proposing code that calls `TotallyNewThing.run()` (a class not in the graph at all) -> `UNKNOWN` -> **never blocked**, even in strict mode. This pair is the single clearest illustration of the "graph is incomplete, don't punish the unknown" design principle.

8. Deep Dive: The Synapse VS Code Extension

Architecture in one sentence: the extension is a thin TypeScript client (~1,200 lines across 12 files) with **no server, no MCP, no persistent process** - every action shells out to the Python CLI with

`child_process.spawn("python", ["-m", "graphify", ...])`, parses stdout / reads `graphify-out/graph.json` off disk, and updates the UI. This is confirmed directly in `cliService.ts` - there is exactly one function (`runGraphify`) that all others funnel through.

8.1 Backend bootstrap

On first activation, `ensureProvisioned()` checks for a private virtual environment at `~/.synapse/vscode-venv` (a short, fixed path chosen specifically to dodge Windows' 260-character `MAX_PATH` limit, which the team's own testing showed broke real extraction runs); if missing, it finds a system Python 3.10+ and creates one, then `pip` installs the package. Note: the package name in this install command is **graphifyy** (double "y") - matching `pyproject.toml`'s actual PyPI package name, distinct from the `graphify` import/CLI name. The extension's own README acknowledges this package may not be published yet and provides a `synapse.devPackagePath` setting to point at a local checkout instead - worth clarifying deliberately if asked, since it looks like a typo at first glance.

8.2 UI surfaces

Surface	What it shows	Backed by
Dashboard (default sidebar view)	Node/edge/community counts, top god node, one-click actions (Rebuild, Update, Visualize, Query, Explain)	Reads <code>graph.json</code> directly; static HTML with <code>command: links</code> , no JavaScript at all (<code>enableScripts: false</code>)
Sidebar tree ("Knowledge Graph")	Two sections: God Nodes (top-10 by degree) and Communities (grouped, capped at 200 rendered nodes per community)	<code>graphModel.ts</code> parses <code>graph.json</code> ; clicking a node jumps straight to its source line
Graph visualization	Interactive drag/zoom node-edge diagram	Reads the already-generated <code>graphify-out/graph.html</code> off disk and re-brands it (green theme, neuron icon) - the extension does not render the graph itself
Docs panel	Static in-editor manual (settings table, troubleshooting)	Pure static HTML, no live data
Copilot Chat tools	<code>synapse_query</code> (natural-language question -> relevant graph nodes) and <code>synapse_explain</code> (symbol -> its direct neighbors)	Registered via VS Code's Language Model Tools API; both shell out to the same CLI path as the manual commands and return raw CLI text output, unformatted, to Copilot
Onboarding walkthrough	6 guided steps (build -> dashboard -> explore -> visualize -> ask -> Copilot)	<code>walkthroughs/*.md</code> , wired via <code>package.json</code>

8.3 Notable implementation decisions

- **Auto-build on open / auto-update on save:** if no graph exists yet, one is built automatically on workspace open (if trusted); saving a `.py` file triggers a debounced (2s) incremental AST-only update. A

failed auto-update is treated as **non-fatal by design** - the last-known-good graph stays visible, the error only appears in the output log, not as a popup. This is a deliberate UX choice (don't interrupt someone's typing over a background refresh failing) but worth knowing, since it means a broken update can go unnoticed unless someone checks the log.

- **Single workspace root, Python-only, untrusted-workspace-blocked** are explicit, documented v1 scope limits, not oversights.
 - **--no-label on the auto-generated visualization**: the extension deliberately runs `graphify cluster-only --no-label` (rather than a full extract with LLM community naming) specifically to preserve the "no API key required" promise - so out-of-the-box community names fall back to their hub node's name rather than an LLM-synthesized theme.
-

9. Codebase Walkthrough

Organized by what matters for understanding the architecture - not a flat file dump.

9.1 Core pipeline (original Graphify)

File	Responsibility	Origin
<code>graphify/ detect.py</code>	File discovery, classification, secret filtering, <code>.gitignore</code> handling, incremental change detection	Original
<code>graphify/ extract.py</code>	Extraction dispatch + all cross-file resolution (import resolution, INFERRED call-graph pass, ID canonicalization)	Original
<code>graphify/ extractors/ engine.py</code>	Shared tree-sitter-driven engine powering ~15 of the 25+ language extractors	Original
<code>graphify/ extractors/*.py (rust, go, dart, ...)</code>	Per-language extractors, some hand-written (Rust), some engine-parameterized	Original
<code>graphify/ build.py</code>	Assembles the NetworkX graph; node/edge dedup; AST-vs-semantic ID reconciliation; cross-language edge guard	Original
<code>graphify/ dedup.py, graphify/ _minhash.py</code>	Fuzzy entity deduplication for non-code nodes	Original
<code>graphify/ cluster.py</code>	Build-time Leiden/Louvain community detection	Original
<code>graphify/ analyze.py</code>	God nodes, surprising connections, suggested questions, import cycles	Original
<code>graphify/ report.py</code>	Renders <code>GRAPH_REPORT.md</code> (function is <code>generate()</code>)	Original
<code>graphify/ export.py, graphify/ exporters/</code>	<code>graph.json</code> , HTML/SVG visualization, Obsidian vault, Neo4j/FalkorDB/Cypher	Original
<code>graphify/ cache.py</code>	SHA-256 content-hash caching (AST + semantic, separately versioned)	Original
<code>graphify/ security.py</code>	SSRF-guarded fetch, path-traversal guard, label sanitization	Original
<code>graphify/ validate.py</code>	Extraction schema enforcement	Original

<code>graphify/ids.py</code>	The single canonical ID-normalization function (<code>make_id</code>)	Original
<code>graphify/llm.py</code>	Multi-backend LLM client for the semantic extraction pass	Original
<code>graphify/cli.py</code>	Command dispatch (<code>if/elif</code> over <code>sys.argv</code> , not <code>argparse</code> subparsers) - home of the <code>extract</code> command's ~1000-line pipeline orchestration	Original, retrofitted with our new commands (<code>reason</code> , <code>validate-claims</code> , <code>preexec-check</code> , <code>semantic-graph</code>) and flags (<code>--weighted</code> , <code>--community-aware</code>) on existing commands
<code>graphify/serve.py</code>	The MCP server (separate entry point <code>graphify-mcp</code> , not a CLI subcommand)	Original, retrofitted - imports and uses our <code>weighted/community-aware</code> retrieval directly, plus exposes our two new tools
<code>graphify/watch.py</code>	Filesystem watcher; incremental AST-only rebuild engine	Original

9.2 Our backend contributions

File	Responsibility
<code>graphify/weights.py</code>	<code>RelationWeightRegistry</code> - the single source of truth for edge importance
<code>graphify/weighted_retrieval.py</code>	Weighted BFS/DFS neighborhood expansion + Dijkstra-based shortest path
<code>graphify/community_detection.py</code>	Pluggable community-detection strategies (Louvain/Leiden/Label-Propagation/Auto), query-time
<code>graphify/community_index.py</code>	Fast lookup structure over communities, preferring stored <code>graph.json</code> attributes
<code>graphify/community_retrieval.py</code>	Confidence-scored community selection + subgraph restriction, with full-graph fallback
<code>graphify/semantic_extraction.py</code>	LLM prompt construction/batching for the semantic-graph pass
<code>graphify/semantic_relations.py</code>	Parses/validates the LLM's semantic-relation JSON response
<code>graphify/semantic_graph.py</code>	Merges validated semantic relations into the live graph (new edges vs. secondary-relation augmentation)
<code>graphify/graph_checks.py</code>	Shared structural fact-checking library (<code>SUPPORTED/CONTRADICTED/UNKNOWN</code>)
<code>graphify/claims.py</code>	Typed, graph-checkable claim representation + parsing from free-form LLM text
<code>graphify/bidirectional_reasoner.py</code>	The claim -> validate -> revise reasoning loop
<code>graphify/preexec_validate.py</code>	Validates proposed code writes against the graph; Claude Code PreToolUse hook integration

9.3 The VS Code extension

File	Responsibility
------	----------------

<code>vscode-extension/src/extension.ts</code>	Activation entrypoint; command registry; auto-build/auto-update wiring
<code>vscode-extension/src/cliService.ts</code>	The single choke point for every Python CLI invocation
<code>vscode-extension/src/dashboardViewProvider.ts</code>	Dashboard webview (stats + quick actions)
<code>vscode-extension/src/graphTreeProvider.ts</code>	Sidebar tree (God Nodes / Communities)
<code>vscode-extension/src/graphWebview.ts</code>	Graph visualization webview (re-brands the generated <code>graph.html</code>)
<code>vscode-extension/src/docsWebview.ts</code>	Static in-editor documentation panel
<code>vscode-extension/src/lmTools.ts</code>	The two GitHub Copilot Chat Language Model Tools
<code>vscode-extension/src/graphModel.ts</code>	TypeScript-side <code>graph.json</code> parsing (degree, top nodes, community grouping)
<code>vscode-extension/src/statusBar.ts, workspaceUtils.ts, logger.ts, brandAssets.ts</code>	Supporting UI/utility modules

9.4 Eval & tests

File	Responsibility
<code>tools/eval/retrieval_accuracy.py</code>	Precision@10/Recall@10/MRR harness (graphify query vs. naive grep baseline), no LLM calls
<code>tools/eval/crosscodeeval.py</code>	Real-LLM code-completion benchmark (no-context vs. BM25 vs. graphify-context)
<code>docs/eval/ACCURACY_REPORT.md</code>	The original 2-way accuracy study
<code>docs/eval/SYNAPSE_VS_GRAPHIFY.md</code>	The 3-way comparison (no graph vs. existing Graphify vs. Synapse)
<code>tests/test_weighted_retrieval.py, test_community_retrieval.py, test_semantic_graph.py, test_bidirectional_reasoner.py, test_preexec_validate.py</code>	Dedicated tests for each of our four backend features
<code>tests/</code> (167 <code>test_*.py</code> files total)	The full original + extended test suite, one file per concern

10. End-to-End Execution Flow

Scenario: a developer types a natural-language question into Copilot Chat while working in VS Code, with the Synapse extension active and a graph already built.

```
1. User types a question in Copilot Chat (agent mode)
  v
2. Copilot's model decides to invoke the "synapse_query" tool
  File: vscode-extension/src/lmTools.ts -> QueryTool.invoke()
  Input: { question: string }
  v
3. Extension checks workspace trust + that a graph exists
  File: vscode-extension/src/workspaceUtils.ts, cliService.ts (hasGraph)
  v
4. Extension shells out to the Python CLI
  File: vscode-extension/src/cliService.ts -> query()
  Command: python -m graphify query "<question>" --graph graphify-out/graph.json --budget N
  v
5. CLI loads graph.json, resolves query terms to seed nodes
  File: graphify/cli.py (query command) -> graphify/serve.py:_query_graph_text()
  v
6. Retrieval: community-aware scoping (if enabled) narrows the graph,
  then weighted BFS/DFS expands the neighborhood from the seed nodes
  Files: graphify/community_retrieval.py, graphify/weighted_retrieval.py
  v
7. Result subgraph is rendered as plain text
  ("NODE <label> [src=...] " / "EDGE <a> --relation--> <b>")
  File: graphify/serve.py:_subgraph_to_text()
  v
8. Extension returns that raw text as a single LanguageModelTextPart
  File: vscode-extension/src/lmTools.ts
  v
9. Copilot's model reads the returned graph context and composes
  its final answer to the user, now grounded in real call/import
  relationships instead of guessing from the visible file alone
```

Scenario: a developer runs `graphify reason "<question>"` from the terminal (the bidirectional-reasoning feature).

```
1. CLI parses the `reason` command -> graphify/cli.py
2. Retrieve graph context (same weighted + community-aware retrieval as above) -> graphify/bidirectional_reasoner.py:retrieve_graph_context()
3. First LLM call: answer + typed claims -> graphify/llm.py (multi-backend)
4. Parse claims from the LLM's JSON -> graphify/claims.py
5. Validate every claim against the graph -> graphify/graph_checks.py
6. If any claim is CONTRADICTED/UNKNOWN:
   second LLM call with the validation
   report fed back in, asking for a
   correction -> graphify/bidirectional_reasoner.py
7. Re-validate the revised claims
8. Format final answer + evidence appendix -> graphify/bidirectional_reasoner.py:format_final_answer()
9. Printed to terminal / returned via MCP tool "reason_with_graph"
```

11. AI / LLM Components

11.1 Where LLMs are used, and where they are deliberately NOT used

Feature	Uses an LLM?	Notes
AST extraction (Pass 1)	No Never	Pure tree-sitter parsing, \$0 cost
Weighted retrieval	No Never	Pure graph algorithm (Dijkstra / priority-ordered BFS)
Community-aware retrieval	No Never	Pure graph algorithm (community detection + scoring)
Doc/paper/image extraction (Pass 3)	Yes Yes	Original Graphify feature, unchanged by us
Semantic graph	Yes Yes	Our addition; opt-in, separate command
Bidirectional reasoning	Yes Yes	Our addition; the <code>reason</code> command
Pre-execution validation	No No LLM call itself	Validates an <i>already-generated</i> proposal against the graph; it's a check, not a generator

11.2 Supported backends (multi-provider, not Claude-only)

`graphify/llm.py` registers 9 backends: `claude`, `claude-cli` (shells out to a local `claude` CLI process instead of calling an API - \$0 cost), `openai`, `gemini`, `kimi`, `ollama` (local, no API key), `deepseek`, `azure`, `bedrock`. Backend auto-detection picks whichever provider has an API key set in the environment; users can also register custom providers.

Every backend implements the same interface, so our semantic-graph and reasoning features get multi-provider support **for free** - they call `graphify.llm._call_llm()`, the same function the original extraction pipeline uses, rather than writing their own API plumbing.

11.3 Request/response shape (semantic extraction, representative of the pattern)

Request (conceptually, backend-specific in practice): a system prompt instructing the model to extract a knowledge graph following a strict JSON schema, plus a user message containing the file contents wrapped in `<untrusted_source path="..." sha256="...">...</untrusted_source>` tags.

Response schema (from the extraction system prompt, quoted from `graphify/llm.py`):

```
{
  "nodes": [{
    "id": "...",
    "label": "...",
    "file_type": "code|document|paper|image|rationale|concept",
    "source_file": "..."
  }],
  "edges": [{
    "source": "...",
    "target": "...",
    "relation": "...",
    "confidence": "EXTRACTED|INFERRED|AMBIGUOUS",
    "confidence_score": 1.0,
    "source_file": "..."
  }],
  "hyperedges": [...],
  "input_tokens": 0,
  "output_tokens": 0
}
```

Error handling: responses are parsed tolerantly (`_parse_llm_json` strips markdown fences and leading/trailing prose); a truncated response (`finish_reason == "length"`, or a syntactically-hollow response that isn't explicitly marked truncated) triggers an adaptive retry that bisects the input chunk and re-sends, up to a bounded retry depth. Our semantic-graph and reasoning features add their own bounded-retry wrapper

(`RetryProvider`) on top, raising a typed error only after retries are exhausted, and both degrade to an explained empty/refusal result rather than crashing if no backend is configured at all.

11.4 Prompt injection defense (a real, cited security control)

Every file's content sent to an LLM is: (1) wrapped in an `<untrusted_source>` tag with a content hash, (2) preceded by an explicit system-prompt instruction never to obey instructions found inside that tag, and (3) passed through `_neutralise_injection_sentinels()`, which regex-defangs known jailbreak/chat-template tokens by inserting a zero-width space mid-token (human-readable, but no longer recognized by any model's own template parser). This matters concretely for a knowledge-graph tool: without it, a malicious comment inside a source file being extracted could attempt to hijack the extraction prompt.

11.5 Cost, latency, rate limits (as actually measured)

Our CrossCodeEval evaluation (see Section 17) used Groq's **free tier**, which enforces a 200,000-token/day account-wide cap - this was hit mid-evaluation and is honestly disclosed in `docs/eval/SYNAPSE_VS_GRAPHIFY.md` rather than papered over (one comparison arm is n=19 instead of n=47 as a result). This is a genuine, citable example of "what happens if the API fails/is rate-limited" for the viva: the team's answer was to disclose the gap and provide a paired-subset comparison across the smaller sample rather than silently dropping the arm.

12. Graph Processing Explained

What a node represents: a code entity (function, class, file, module), a concept extracted from docs/papers, or (new) an LLM-inferred abstract concept with no direct source location.

What an edge represents: a directed, typed, confidence-tagged relationship. Structural types (`calls`, `imports`, `imports_from`, `inherits`, `implements`, `references`, `contains`, `method`) come from AST parsing; semantic types (`semantically_similar_to` for docs/images - original; `handles/validates/manages/responsible_for/related_to` for code - our addition) come from LLM inference.

Graph construction (Section 3.2-3.4 above) - tree-sitter AST walk -> per-file nodes/edges -> cross-file resolution -> NetworkX assembly -> dedup.

Graph traversal - three distinct algorithms now coexist:

1. Original unweighted BFS/DFS (still available via `--unweighted`).
2. Our weighted priority-ordered BFS/DFS (`expand_neighborhood`, default).
3. Our Dijkstra-based shortest path (`find_shortest_path`, weighted by default).

Graph querying: a natural-language question is matched against node *labels* (not full-text search) to find seed nodes, then one of the traversal algorithms above expands outward from those seeds up to a configurable depth, optionally scoped to a community first.

Graph visualization: `export.to_html()` uses vis.js; automatically degrades for very large graphs by recursively building a community-level meta-graph (one node per community) instead of failing outright. `export.to_svg()` uses matplotlib + a spring layout for static, embeddable images. The VS Code extension does not generate visualizations itself - it displays the already-generated `graph.html`.

Graph algorithms used, by name:

Algorithm	Where	Purpose
Tree-sitter incremental parsing	<code>extractors/</code>	Turn source text into an AST
MinHash + Locality-Sensitive Hashing	<code>dedup.py</code> , <code>_minhash.py</code>	Sub-quadratic candidate blocking for fuzzy entity dedup
Jaro-Winkler string similarity	<code>dedup.py</code> (via <code>rapidfuzz</code>)	Verify blocked candidates are genuinely the same entity
Leiden community detection	<code>cluster.py</code> , <code>community_detection.py</code>	Partition the graph into densely-connected communities (a refinement of Louvain that guarantees each detected community is internally connected)
Louvain community detection	<code>cluster.py</code> , <code>community_detection.py</code> (fallback)	Greedy modularity-maximizing clustering
Label propagation	<code>community_detection.py</code> (new, query-time only)	Fast community approximation by iterative majority-vote label spreading
Dijkstra's shortest path	<code>weighted_retrieval.py</code> (our addition)	Cheapest-cost path honoring relation-importance weights

Priority-ordered BFS/ DFS	<code>weighted_retrieval.py</code> (our addition)	Hop-bounded expansion favoring strong relationships
Betweenness centrality	<code>analyze.py</code>	Identify nodes that bridge otherwise-separate parts of the graph, for surprise detection and suggested questions
Cycle detection (<code>simple_cycles</code>)	<code>analyze.py</code>	Find import cycles at the file level

13. Technology Stack

Technology	Why it's used	Where
Python 3.10+	Core engine language; broad library ecosystem for graph/NLP/ML tooling	<code>graphify/</code>
tree-sitter (+ ~25 grammar packages)	Fast, error-tolerant, incremental parsing across many languages with one consistent API	<code>graphify/extractors/</code>
NetworkX	The in-memory graph data structure and algorithm library (shortest path, centrality, cycle detection)	<code>graphify/build.py</code> and throughout
graspologic / Leiden algorithm	Community detection that (unlike plain Louvain) guarantees internally-connected communities	<code>graphify/cluster.py</code> , <code>community_detection.py</code>
rapidfuzz	Fast Jaro-Winkler string similarity for entity dedup	<code>graphify/dedup.py</code>
A custom MinHash/LSH implementation	Sub-quadratic dedup candidate blocking, without pulling in <code>datasketch</code> (which has a Windows hang bug via a transitive <code>scipy</code> import)	<code>graphify/_minhash.py</code>
Multi-provider LLM clients (Anthropic, OpenAI SDK used generically, Google Gemini, Ollama, ...)	Semantic extraction, semantic-graph edges, and bidirectional reasoning; kept provider-agnostic so users aren't locked to one vendor	<code>graphify/llm.py</code>
MCP (Model Context Protocol)	Standardized way to expose graph-query tools to any compatible AI client, not just this project's own CLI/extension	<code>graphify/serve.py</code>
TypeScript + VS Code Extension API	The Synapse editor integration	<code>vscode-extension/src/</code>
VS Code Language Model Tools API	Registers <code>synapse_query/</code> <code>synapse_explain</code> as tools GitHub Copilot Chat can call autonomously	<code>vscode-extension/src/</code> <code>lmTools.ts</code>
vis.js	Interactive in-browser graph visualization (loaded by the generated <code>graph.html</code>)	<code>graphify/exporters/html.py</code>
OpenAI Python SDK (generic client)	Used against Groq's free-tier OpenAI-compatible endpoint for the CrossCodeEval benchmark	<code>tools/eval/</code> <code>crosscodeeval.py</code>
pytest	Test runner for the 167-file test suite	<code>tests/</code>
uv	Python dependency/environment management	repo root

Likely viva questions on this list, answered concretely in Section 20.

14. Architecture Diagrams

Diagram 1 - Overall System Architecture (Synapse today)

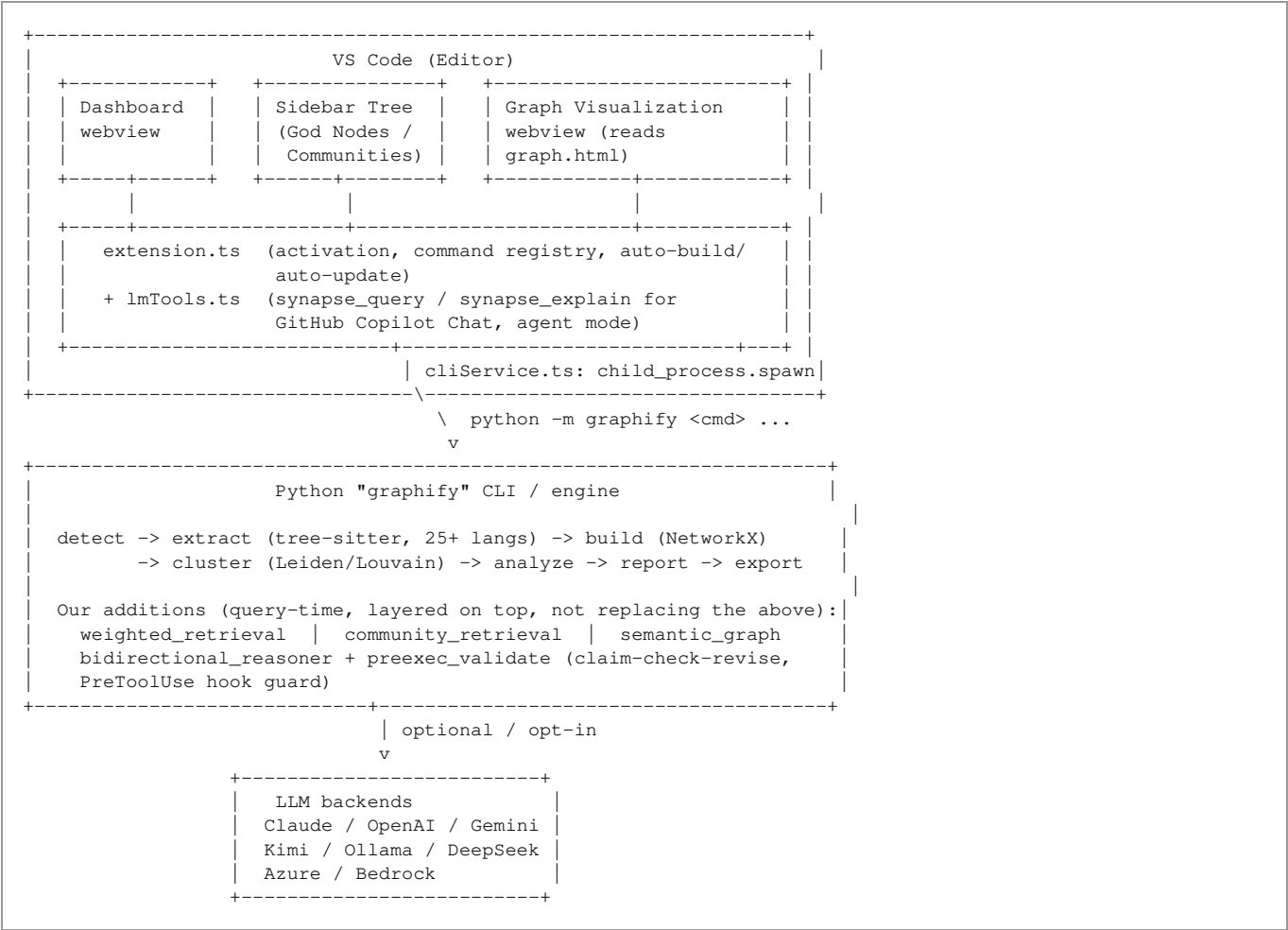
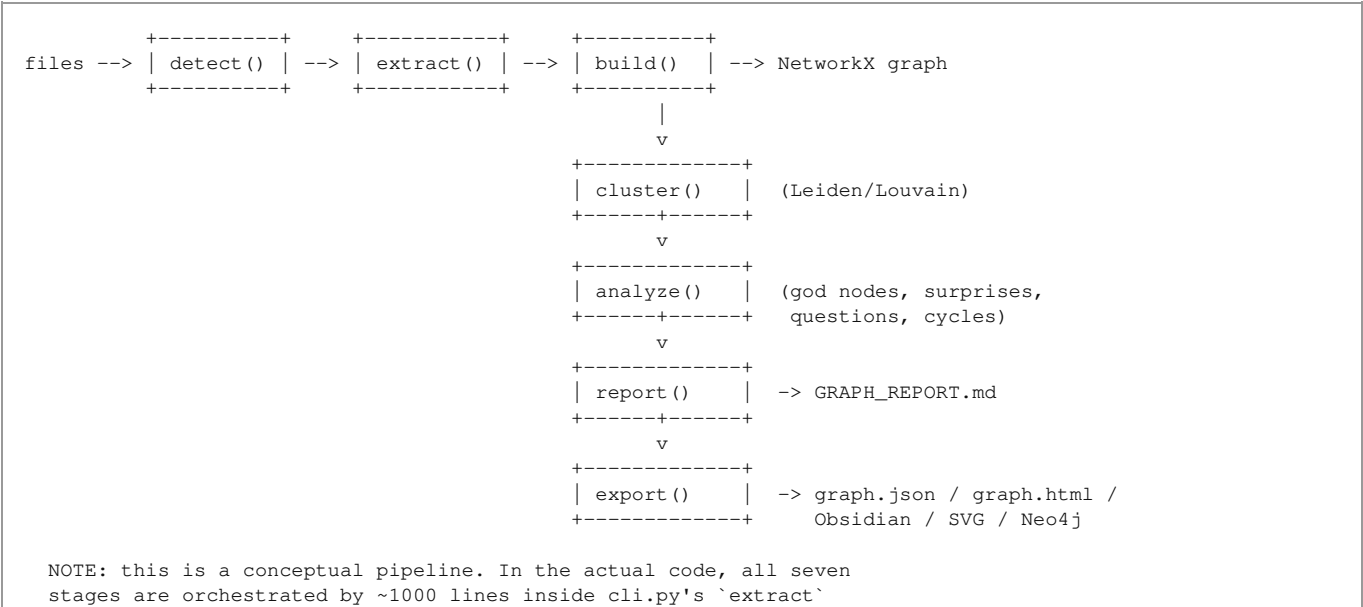


Diagram 2 - Original Graphify Pipeline



command handler -- there is no single build_graph()/analyze()/export() function to point to. See Section 3.1.

Diagram 3 - Query-Time Architecture (where our features plug in)

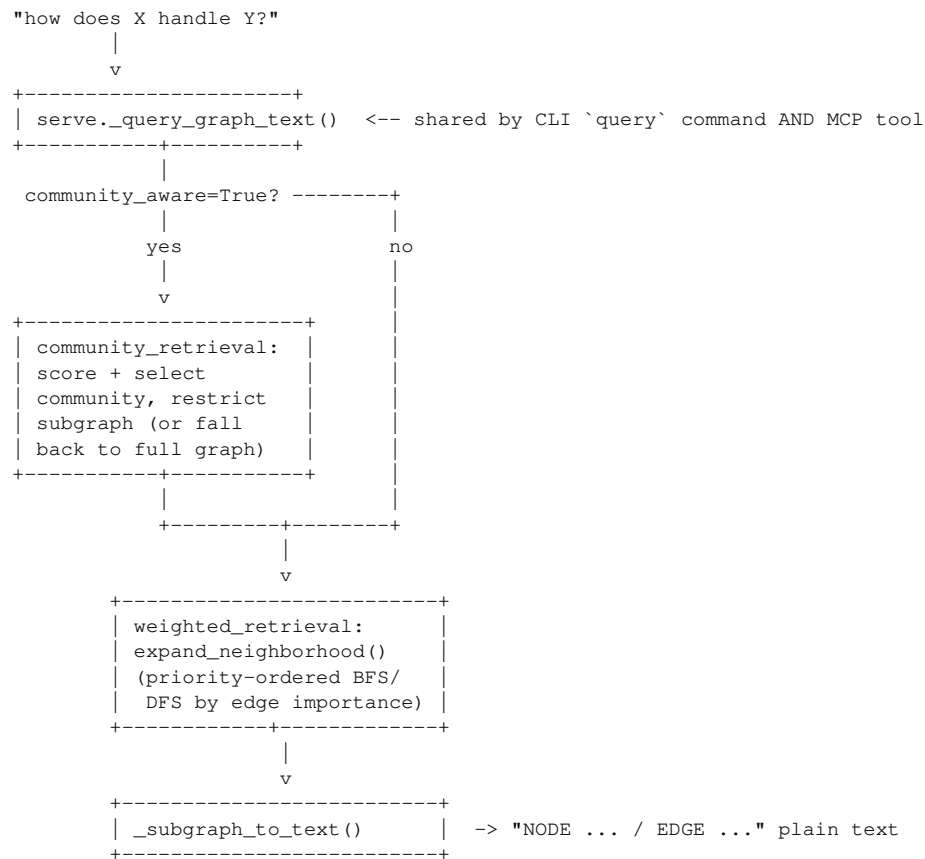
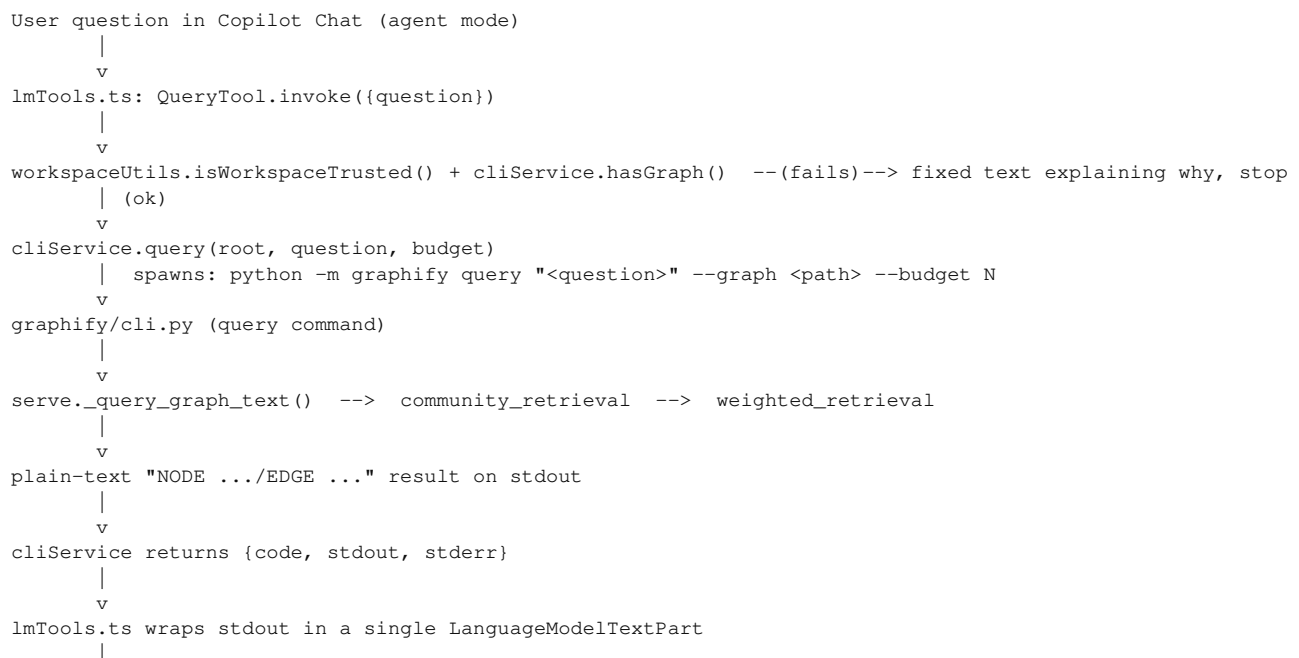


Diagram 4 - End-to-End Request Flow (Copilot Chat -> Synapse -> back)



v
Copilot's model reads it as tool output and writes the final answer

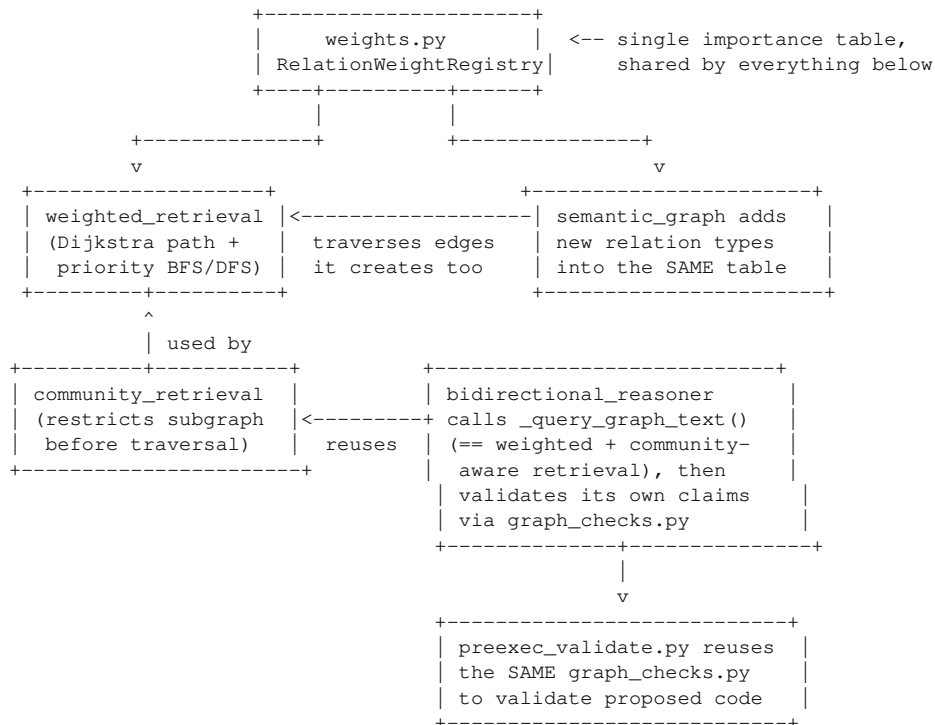
Diagram 5 - LLM / API Interaction (semantic extraction & reasoning, shared pattern)

```

file/prompt content
|
v
wrap in <untrusted_source path=".." sha256="..">...</untrusted_source>
|
v
_neutralise_injection_sentinels() (defang <|im_start|>, [INST], forged closers)
|
v
graphify/llm.py: backend-agnostic _call_llm(backend, model, system, messages)
|
+--> claude / openai / gemini / kimi / ollama / deepseek / azure / bedrock / claude-cli
|
v
raw text response
|
v
_parse_llm_json() (strip markdown fences, tolerate leading/trailing prose)
|
truncated/hollow? --(yes)--> bisect chunk, retry (bounded depth)
| (no)
v
typed, validated result (nodes/edges, OR semantic relations, OR claims)
|
v
merged into the graph / returned to caller

```

Diagram 6 - Our Feature Interconnections



15. Important Code Snippets

1. The importance-to-cost conversion that makes weighted retrieval "just Dijkstra" ([graphify/weights.py](#))

```
def importance_to_cost(importance: float) -> float:
    return 1 / max(importance, 1e-6)
```

Why it matters: this one line is the entire bridge between "a curated table of how important each relation type is" and "something NetworkX's built-in `shortest_path(weight="weight")` can use," because Dijkstra minimizes cost, and we want to maximize importance - so cost is defined as the inverse.

2. The safety-biased aggregate verdict ([graphify/graph_checks.py](#))

```
# any CONTRADICTED -> overall CONTRADICTED
# all SUPPORTED -> overall SUPPORTED
# a mix of SUPPORTED and UNKNOWN (no contradictions) -> still UNKNOWN, for safety
```

Why it matters: this is the single design decision that makes the reasoning feature trustworthy rather than falsely confident - it would rather under-claim than over-claim.

3. The semantic-edge augmentation rule ([graphify/semantic_graph.py](#), paraphrased)

```
if graph.has_edge(source, target):
    append_to_secondary_relations(edge, new_relation)    # never overwrite
else:
    graph.add_edge(source, target, relation=new_relation, _origin="semantic_code")
```

Why it matters: without this check, applying the semantic-graph pass on top of an existing structural edge would silently destroy the original `relation` field.

4. The preexec blocking rule ([graphify/preexec_validate.py](#), paraphrased)

```
blocked = strict_mode and (overall_verdict == CONTRADICTED)
# UNKNOWN never blocks, even in strict mode
```

Why it matters: the single line that prevents the validation hook from being annoying - it only stops a *provably wrong* claim, never merely-unrecognized code.

5. The cross-language phantom-edge guard ([graphify/build.py](#), paraphrased)

```
if relation == "calls":
    if confidence == "INFERRED" and src_family and tgt_family and src_family != tgt_family:
        drop_edge()    # an EXTRACTED cross-language call (e.g. FFI) is still trusted
else: # imports / imports_from / references
    if src_family and tgt_family and src_family != tgt_family:
        drop_edge()    # stricter: any known-but-different family is dropped
```

Why it matters: this is what stops a bare `render()` call in a Python file from spuriously resolving to a same-named function in an unrelated JavaScript file - a real bug class the comment cites by issue number.

6. Content-salted cache key (`graphify/cache.py`, paraphrased)

```
file_hash = SHA256(file_bytes + b"\x00" + relative_path.lower())
```

Why it matters: two files with identical content at two different paths must not collide in the cache - the path is deliberately mixed into the hash.

16. Dependencies & Environment / How to Run

16.1 Core dependencies (pyproject.toml)

- **Package name:** `graphifyy` (note the double "y" - distinct from the `graphify` import/CLI name). Version `0.9.27`. Requires Python `>=3.10`.
- **Core deps:** `networkx`, `numpy`, `rapidfuzz`, `~25 tree-sitter-*` grammar packages.
- **Optional extras:** `mcp` (MCP server support), `neo4j/falkordb` (graph DB export), `pdf`, `watch`, `svg`, `leiden` (`graspologic`), `office` (`python-docx/openpyxl`), `video` (`faster-whisper`), one extra per LLM provider (`openai`, `anthropic`, `gemini/kimi/ollama`, `bedrock`), plus an `all` extra.
- **Entry points:** `graphify` (the CLI) and `graphify-mcp` (the MCP server - a separate program, see Section 3.6).

16.2 VS Code extension dependencies

No runtime dependencies at all - only devDependencies (TypeScript, mocha, `@vscode/test-electron`, `@vscode/vsce`). This confirms the extension is a thin wrapper: it ships no bundled graph/LLM libraries, it shells out to the Python CLI for everything.

16.3 Environment variables (selected, all read directly via `os.environ/os.getenv`, no `.env.example` exists in the repo)

Variable	Purpose
<code>ANTHROPIC_API_KEY</code> , <code>OPENAI_API_KEY</code> , <code>GEMINI_API_KEY</code> / <code>GOOGLE_API_KEY</code> , <code>MOONSHOT_API_KEY</code> (Kimi), <code>DEEPSEEK_API_KEY</code> , <code>AZURE_OPENAI_API_KEY</code>	LLM backend credentials
<code>OPENAI_BASE_URL</code>	Points the OpenAI-compatible client at a different provider (used to hit Groq's free tier in our eval scripts)
<code>GRAPHIFY_PREEEXEC</code> , <code>GRAPHIFY_PREEEXEC_STRICT</code>	Enable / strict-mode the pre-execution validation hook (opt-in, off by default)
<code>GRAPHIFY_OUT</code>	Override the <code>graphify-out</code> directory name
<code>GRAPHIFY_MAX_GRAPH_BYTES</code>	Cap on graph file size accepted (memory-bomb guard)

16.4 How to run the project end-to-end

```
# 1. Clone and install (Python side)
git clone <repo-url> && cd Synapse
uv sync                                # add --extra anthropic / --extra all as needed

# 2. Build a graph (AST-only, no API key needed)
uv run graphify extract . --code-only
```

```
# 3. Query it
uv run graphify query "how does the CLI dispatch commands?" --graph graphify-out/graph.json
uv run graphify query "..." --graph graphify-out/graph.json --community-aware

# 4. Try the new reasoning feature (needs an LLM backend configured)
uv run graphify reason "does LoginHandler inherit from SessionStore?"

# 5. Run the test suite
uv run pytest tests/ -q

# 6. VS Code extension
cd vscode-extension
npm install && npm run compile
npm run test:unit          # fast, no VS Code binary needed
npm run package            # produces a .vsix
```

17. Accuracy Evaluation - What We Measured

Two harnesses, both documented in `docs/eval/` - read those files directly for full methodology; summarized here.

17.1 Retrieval accuracy (graph traversal only, no LLM)

20 hand-written questions with known-correct targets, scored with Precision@10 / Recall@10 / MRR:

Method	Precision@10	Recall@10	MRR
No Graphify (naive grep)	0.083	0.75	0.318
Existing Graphify (pre-our-changes, plain BFS)	0.050	0.45	0.251
Synapse (weighted + community-aware)	0.075	0.70	0.271

Honest finding: on this narrow, single-repo precision metric, plain keyword grep still edges out both graph-based approaches. This is expected and explained in the report: graphify's traversal is built to hand an LLM *broad* supporting context, not to be a precision-ranked top-k search engine on its own - and our weighted+community-aware changes measurably close most of the gap versus the pre-existing engine without fully closing it.

17.2 CrossCodeEval - real LLM code completion (the metric that matters most)

47 tasks from two real GitHub repositories, three completions generated per task via Groq's free tier (`qwen/qwen3.8-27b`), scored by Exact Match and edit similarity:

Condition	Exact Match	Edit Similarity
No context	6.4%	0.533
BM25 context (standard baseline)	12.8%	0.595
Synapse context	29.8%	0.689

Paired comparison (same 19 tasks, all four conditions - Existing Graphify's full n=47 run hit Groq's daily token cap mid-evaluation, disclosed rather than hidden):

Condition	Exact Match	Edit Similarity
No Graphify	0.0%	0.433
BM25 context	15.8%	0.637
Existing Graphify	21.1%	0.672
Synapse	31.6%	0.734

Bottom line: on the metric that reflects real downstream usefulness (does the retrieved context actually help an LLM write correct code), our retrieval additions produced a measurable **~10 percentage point** improvement in exact-match accuracy over the pre-existing engine, and both graph-based approaches comfortably beat no-context and BM25.

17.3 Regression testing

After merging the three retrieval features (weighted, community-aware, semantic graph), reported in `docs/eval/SYNAPSE_VS_GRAPHIFY.md`: full suite **3,732 passed, 42 failed, 46 skipped**. All 42 failures were re-run against the pre-merge codebase in an isolated worktree - 41 failed identically there too (pre-existing Windows-only issues: symlink handling, long paths, `.gitignore` edge cases). The 1 remaining was a test-harness false positive caused by this machine's project folder containing spaces, confirmed identical on old and new code from a space-free path. **Net new regressions from that merge: 0.**

A fresh full run performed during this analysis (after also merging the reasoning/preexec-validation feature from `origin/main`, in this session's sandboxed shell): **3,743 passed, 49 failed, 46 skipped**. The 7 additional failures beyond the previously-known 42 are entirely in `tests/test_security.py` (`test_validate_url_accepts_http/https`, `test_safe_fetch_*`), all raising `ValueError: DN...` (truncated - consistent with a DNS-resolution error). These tests call `graphify.security`'s SSRF-guard, which performs a real DNS lookup before validating a URL; this session's Bash tool sandbox very likely has no outbound network/DNS access, which would make these fail regardless of any code change. The other 42 failures match the same previously-documented categories exactly by test name (symlink handling in `test_detect.py/test_extract.py/test_image_vision.py`, path-length/`.gitignore` edge cases, and install-path issues plausibly related to this machine's folder path containing spaces). **This was not re-verified against an isolated pre-merge worktree in this session** (that comparison, and confirming the network-sandbox theory for the 7 new failures, is listed in Section 23 as a task for the team to complete in a normal, non-sandboxed environment before citing a final "0 regressions" claim for the reasoning/preexec-validation merge).

18. Limitations & Future Work

18.1 Currently implemented (do not overclaim these as future work)

- Weighted retrieval, community-aware retrieval, semantic graph, bidirectional reasoning, and pre-execution validation are all implemented, tested, and wired into the CLI/MCP server.
- The VS Code extension's Dashboard, sidebar, graph visualization, Copilot Chat tools, and onboarding walkthrough are all implemented and have passing unit tests.
- Both accuracy harnesses are implemented and have produced real, disclosed numbers.

18.2 Known limitations (from the actual implementation, not guessed)

- **Grep still wins on narrow precision@10** on a small single-repo sample (Section 17.1) - graph retrieval's advantage shows up in downstream LLM task accuracy, not in isolated top-k precision.
- **Community-aware retrieval falls back to the full graph** whenever confidence is low or seeds span multiple communities - a safety choice, but it means the feature provides no benefit on ambiguous queries.
- **Semantic-graph's secondary-relations design is a documented workaround**, not the final intended architecture - a true multigraph representation (parallel edges) was probed but not completed.
- **Pre-execution validation's non-Python support is heuristic-only** (regex-based, not real AST parsing) - Python is the only language with first-class support.
- **The CrossCodeEval evaluation used a free-tier, non-frontier model** (`qwen/qwen3.8-27b`) and hit a real rate limit mid-run for one comparison arm ($n=19$ instead of $n=47$) - disclosed, not hidden, but a real sample-size caveat.
- **Ground truth for the retrieval harness is hand-labeled by the team itself**, not independently audited.
- **The self-graph at `graphify-out/` in this repository is stale** - it was built before any of the four backend features existed and should be refreshed (`graphify update .`) before being used to explore the current codebase with AI assistance.
- **VS Code extension is single-workspace-root, Python-only, and disabled on untrusted workspaces** - all documented v1 scope limits.
- **The extension's declared PyPI package name (`graphifyy`) may not be published** - a `devPackagePath` setting exists as a workaround.

18.3 Possible future work (explicitly NOT implemented yet - from the team's own research docs)

- Learning edge weights from query-log feedback instead of a fixed importance table.
- Query-type-conditioned weight profiles and cost-bounded (not just hop-bounded) traversal.
- Persisting the community index alongside `graph.json` for cold-start speed; multi-resolution (parent/child) community hierarchies.
- Feeding real source-code snippets (not just labels) into the semantic-graph LLM prompt; per-file caching of semantic extraction.
- A true multigraph representation so structural and semantic edges become genuine parallel edges.
- Extending pre-execution validation with real AST-based parsers for non-Python languages.
- Multi-root workspace support and non-Python auto-update triggers in the VS Code extension.

- A full n=47 CrossCodeEval run for the "Existing Graphify" arm once API quota allows (currently n=19).
-

19. How to Explain Our Contributions During the Presentation

Contribution 1 - Weighted Retrieval

Problem: the original tool treats a "calls" relationship and an "imports" relationship as equally important when searching the graph, which loses real signal about what's actually relevant to a question.

Existing Graphify approach: unweighted breadth-first/depth-first search - every edge is one hop, full stop.

Our approach: a central importance table for every relation type, converted to a traversal cost, used two ways: real Dijkstra for point-to-point shortest paths, and priority-ordered hop-bounded expansion for open-ended neighborhood queries.

Implementation: `graphify/weights.py` + `graphify/weighted_retrieval.py`, default-on for `query/path`.

Result/benefit: measurably improved downstream LLM code-completion accuracy (see Section 17.2); a concrete example we can show live - given a choice between a 1-hop weak import and a 2-hop strong call chain, weighted retrieval takes the extra hop for the stronger relationship.

How to say it (30-60 sec): *"The original tool's graph search treated every relationship the same - a function call and a stray import counted equally. We built a weighting system that ranks relationships by real importance - calls matter more than imports - and used that to power both shortest-path search and neighborhood exploration. In our benchmark, this measurably improved how often an LLM using our retrieved context wrote the exact correct code, compared to the original unweighted engine."*

Contribution 2 - Community-Aware Retrieval

Problem: even with weighted edges, search still spans the entire graph, so a few incidental cross-module edges can pull results into unrelated territory on a large codebase.

Existing Graphify approach: no notion of "stay within the relevant part of the codebase" - always searches everything.

Our approach: detect communities (clusters) at query time, score which community a query's seed nodes belong to with a confidence score, and restrict the search to that community - but fall back to the full graph automatically if confidence is too low, rather than risk missing something.

Implementation: `graphify/community_detection.py`, `community_index.py`, `community_retrieval.py`; opt-in via `--community-aware`.

Result/benefit: demonstrated in tests to correctly exclude an unrelated module (e.g., a database module) when a query is clearly about authentication, while safely falling back to the full graph when a query legitimately spans modules.

How to say it (30-60 sec): *"On a large codebase, the graph naturally forms clusters - an authentication module, a database module, and so on. We added a feature that first figures out which cluster a question is really about, with a measurable confidence score, and searches there first - but if it's not confident, it safely falls back to searching everything, so we never silently miss the right answer just to be fast."*

Contribution 3 - Semantic Graph

Problem: structural relationships (`calls`, `imports`) tell you *what* connects to *what*, but never *why* - no sense

of responsibility or intent.

Existing Graphify approach: LLM-based semantic extraction existed, but only for documents/papers/images - code was excluded because AST already covers structure.

Our approach: an opt-in pass that asks an LLM to label code relationships with a fixed, closed vocabulary of intent - `handles`, `validates`, `manages`, `responsible_for`, `related_to` - and merges them into the graph without ever overwriting an existing structural relationship.

Implementation: `graphify/semantic_extraction.py`, `semantic_relations.py`, `semantic_graph.py`; separate CLI command `graphify semantic-graph`.

Result/benefit: adds an intent layer that's directly usable by the same weighted/community-aware retrieval, with no additional code needed there.

How to say it (30-60 sec): *"AST parsing tells you a function calls another function, but not why. We added an optional pass where an LLM reads the code and labels relationships with intent - like 'this function validates that one' - using a fixed vocabulary so results stay consistent. The tricky part was making sure this never silently overwrites what the AST already knows for certain - so we designed it to only add new information alongside the existing structural facts, never replace them."*

Contribution 4 - Bidirectional Reasoning + Pre-Execution Validation

Problem: an LLM reading graph context can still hallucinate a wrong answer; separately, an AI coding agent can propose code that calls a method that doesn't exist.

Existing Graphify approach: none - graph context is handed to the LLM, and whatever it says is final.

Our approach: (1) have the LLM emit its answer *plus* typed, checkable claims; verify every claim against the graph with a deliberately conservative verdict system (unknown is never treated as false, but also never counted as fully verified); if a claim is contradicted, send the evidence back to the LLM for a self-correction pass; refuse to present an answer as fact if it's still wrong after that. (2) Separately, hook into an AI coding agent's *pre-execution* step to block a proposed code edit only when it's provably contradicted by the graph - never merely because a symbol is unrecognized.

Implementation: `graphify/graph_checks.py` (shared verification core), `claims.py`, `bidirectional_reasoner.py`, `preexec_validate.py`.

Result/benefit: a concrete, demonstrable self-correction (an LLM wrongly claims class A inherits from B; the graph shows it actually inherits from C; the system catches the contradiction and gets the LLM to correct itself before showing the user anything).

How to say it (30-60 sec): *"Even with good graph context, an LLM can still say something the graph directly contradicts. We built a verification loop: the LLM has to state its claims explicitly, we check each one against the graph, and if something's wrong, we send the evidence back and make it correct itself - and if it still can't get it right, we refuse to present it as a confident answer. We also applied the same checking logic to a completely different problem: stopping an AI coding assistant from writing code that calls a method we can prove doesn't exist, before that code is ever written - without blocking it for things the graph simply doesn't know about, since that's not the same as being wrong."*

Contribution 5 - Synapse VS Code Extension

Problem: the original tool was CLI-only, which is a real barrier for anyone who isn't comfortable in a terminal,

and doesn't integrate with the AI coding assistant most developers already use daily.

Existing Graphify approach: none - no editor integration at all.

Our approach: a from-scratch VS Code extension with a Dashboard, an explorable sidebar (God Nodes/Communities), an interactive graph visualization, an onboarding walkthrough, and - most importantly - two tools registered with VS Code's Language Model Tools API so GitHub Copilot Chat can call into the graph automatically, in agent mode, without the user doing anything manual.

Implementation: `vscode-extension/` (TypeScript), a thin client that shells out to the same Python CLI - no server, no duplicated logic.

Result/benefit: makes every other contribution actually usable day-to-day inside the editor, and gets the graph in front of Copilot Chat with zero manual steps once a graph exists.

How to say it (30-60 sec): *"All of our backend improvements only matter if people actually use them. We built a VS Code extension that gives you a dashboard, an explorable view of the codebase's most important pieces, an interactive graph, and - the part that matters most - it registers itself as a tool GitHub Copilot Chat can call on its own. So when you ask Copilot a question about your code, it can automatically pull in real graph context from our engine before answering, without you doing anything extra."*

20. Viva Preparation - Full Q&A Bank

Basic Questions

Q: What is Graphify, in one sentence? A: A tool that parses a codebase (and its docs/papers/images) into a knowledge graph of typed, confidence-tagged relationships, so questions about the code can be answered by walking a graph instead of re-reading raw files. *Short viva answer:* "It turns a codebase into a queryable graph of code relationships."

Q: What is Synapse? A: Our fork of Graphify, adding weighted and community-aware retrieval, an LLM-based semantic-intent layer, a claim-verification reasoning loop with a code-write safety hook, and a full VS Code extension with GitHub Copilot Chat integration. *Short viva answer:* "Graphify's engine plus four new retrieval/verification features and a VS Code extension we built."

Q: What problem does this project solve? A: LLMs and developers need structural context about a codebase to answer questions accurately; dumping raw files is expensive and imprecise, and keyword search is structurally blind. This project builds and queries a graph instead.

Q: Why a graph and not just a search index? A: A graph preserves *relationships* (this function calls that one, this class inherits from that one) that a plain text/keyword index discards - which is exactly the information needed to answer "how does X work" style questions, not just "where does X appear."

Technical Questions

Q: How does extraction work, technically? A: Each file is parsed by a language-specific tree-sitter grammar; a recursive AST walk (imperative dispatch on node type, not tree-sitter's query language) emits nodes for declarations and edges for relationships; a corpus-wide second pass then resolves cross-file calls that couldn't be resolved from a single file's AST alone, guarded against cross-language false matches.

Q: How are duplicate nodes prevented? A: Three layers - within-file dedup via a `seen_ids` set, exact-ID collision handling in NetworkX (last writer wins, semantic nodes intentionally added after AST nodes), and a fuzzy MinHash+LSH+Jaro-Winkler pipeline for non-code labels only (code identity is always the fully-qualified ID, never merged by label).

Q: What's the difference between weighted retrieval's shortest-path and its neighborhood expansion? A: Shortest-path (`find_shortest_path`) genuinely uses Dijkstra's algorithm via NetworkX's weighted `shortest_path`. Neighborhood expansion (`expand_neighborhood`) is not Dijkstra - it's the original hop-bounded BFS/DFS, but within each hop, candidates are sorted by descending edge importance before being added to the next frontier, so hop-count semantics stay identical to the old unweighted version.

Q: How does community-aware retrieval decide whether to restrict the search? A: It computes a confidence score (`gap_confidence` x `seed_coverage`, where `gap_confidence` compares the top-scoring community against the second-best). If confidence clears a threshold (default 0.55), the search is restricted to that community's subgraph; otherwise it falls back to the full graph and reports why (`no_communities`, `low_confidence`, `seeds_outside`, etc.).

Q: How does the semantic-graph feature avoid corrupting existing edges? A: A NetworkX edge is keyed only by (`source`, `target`) - calling `add_edge` again would overwrite the `relation` field. So when a structural edge already exists between two nodes, the new semantic relation is appended to a `secondary_relations` list on that edge instead of replacing anything; a brand-new graph edge is only created for pairs with no prior

connection.

Q: What does "bidirectional" mean in bidirectional reasoning? A: The direction of information flow in the reasoning loop, not a graph-search algorithm. Normal GraphRAG is one-directional (graph -> LLM -> answer); this adds the reverse leg (LLM's claims -> graph verification -> feedback to the LLM), which is where "bidirectional" comes from.

Q: How is a "claim" represented internally? A: A small, typed dataclass (`type`, `source`, `target`, `relation`, `symbol`) - deliberately not free-form text, specifically so it can be validated against the graph without needing NLP over the final answer.

Architecture Questions

Q: Draw the high-level architecture. A: See Diagram 1 (Section 14) - VS Code UI surfaces -> `extension.ts/lmTools.ts` -> `cliService.ts` shells out to the Python CLI -> original pipeline + our four retrieval/verification features -> optional LLM backends.

Q: Does the VS Code extension talk to a server, or run its own copy of the logic? A: Neither - it has no server and no duplicated logic. Every action is a one-shot `child_process.spawn` of the Python CLI; results come from parsing stdout or reading `graph.json` off disk. This was confirmed directly by reading `cliService.ts`.

Q: Is graphify serve part of the same CLI as graphify extract? A: No - `serve.py` is a completely separate console-script entry point (`graphify-mcp`), with its own argument parser. It is not reachable through the `graphify` CLI's command dispatch at all.

Q: Why does the extension shell out to a CLI instead of embedding a Python interpreter or reimplementing the logic in TypeScript? A: It avoids duplicating ~15,000 lines of Python graph logic in a second language, keeps the Python engine as the single source of truth, and lets the extension stay a thin, easily-maintained client.

AI/LLM Questions

Q: Which LLM providers does the project support? A: Nine backends: Claude (API and local CLI), OpenAI, Gemini, Kimi, Ollama (local), DeepSeek, Azure, and Bedrock - auto-detected from whichever API key is present in the environment.

Q: Do you need an API key to use the core tool? A: No - AST extraction, weighted retrieval, and community-aware retrieval are all pure algorithms with zero LLM calls. An API key is only needed for the original doc/paper/image extraction pass, and for our opt-in semantic-graph and reasoning features.

Q: How do you defend against prompt injection when feeding source files to an LLM? A: Every file's content is wrapped in a tagged, hash-fingerprinted `<untrusted_source>` block; the system prompt explicitly instructs the model never to obey instructions found inside that block; and known jailbreak/chat-template tokens are regex-defanged before the text is sent.

Q: What happens if the LLM call fails or the response is malformed? A: Parsing is tolerant of markdown fences and stray prose; a truncated or "hollow" response triggers a bounded, chunk-bisecting retry; our semantic-graph and reasoning features add their own bounded-retry wrapper on top and degrade to an explained empty/refusal result rather than crashing if no backend is configured.

Q: What are the cost/latency/rate-limit considerations you actually observed? A: Our CrossCodeEval

evaluation hit Groq's free-tier 200,000-token/day cap mid-run, which we disclosed rather than hid - one comparison arm ended up as $n=19$ instead of $n=47$ as a direct, visible consequence.

Algorithm Questions

Q: Explain the Leiden algorithm in your own words. A: It's a refinement of Louvain community detection that fixes Louvain's known flaw of sometimes producing internally-disconnected communities. It runs local greedy modularity-improving moves, then a refinement phase that guarantees every resulting sub-community is actually connected, then aggregates communities into super-nodes and repeats.

Q: Why Dijkstra for shortest path but not for neighborhood expansion? A: Shortest path has a clear cost-minimization objective Dijkstra solves directly. Neighborhood expansion needs to preserve the original hop-count-bounded API (same depth parameter, same general shape of results as the old unweighted BFS/DFS) - reusing Dijkstra there would change the semantics of "depth," so a priority-ordered BFS/DFS was used instead to keep the API stable while still favoring strong relationships.

Q: What's MinHash/LSH doing here, and why not just compare every pair of labels? A: All-pairs comparison is $O(n^2)$, infeasible on a graph with thousands of nodes. MinHash creates a compact sketch of each label's character-shingle set; LSH buckets similar sketches together so only candidates in the same "band" are ever compared in detail - turning an intractable comparison into a near-linear one.

Code Questions

Q: What does `RelationWeightRegistry.register()` let you do? A: Add a new relation type's importance value at runtime with a single call - no other code (traversal, ranking) needs to be touched, because everything reads from the same central table.

Q: Walk me through `check_relation` in `graph_checks.py`. A: It resolves both claim endpoints (fuzzy symbol matching, up to a handful of candidates each side), checks every endpoint pair for a matching edge (including secondary relations from the semantic-graph feature), and - only for inheritance-like relations, where a node can have at most one real parent - treats an explicit edge to a *different* parent as a genuine contradiction. Everything else that can't be positively matched falls back to `UNKNOWN`.

Q: What does `proposal_from_tool_input` do? A: Normalizes a raw AI-agent tool call (`Write`, `Edit`, `Bash`, etc.) into a standard `ProposedAction` object (kind, language, content, file path) that the rest of the pre-execution validator can work with regardless of which tool produced it.

"Why did you use X?" Questions

Q: Why NetworkX instead of a graph database from the start? A: The core workflow is "build a graph once, run algorithms on it in-process" - NetworkX is a mature, well-documented in-memory graph library with the exact algorithms needed (shortest path, centrality, community detection) built in. Graph database export (Neo4j, FalkorDB) is offered as an *optional* output format for teams that want persistent/queryable storage, not a requirement.

Q: Why tree-sitter instead of each language's own official parser/compiler frontend? A: One consistent parsing API across 25+ languages, error-tolerant (parses incomplete/broken code without failing), and fast enough for large corpora - using each language's own compiler frontend would mean 25+ different APIs and dependencies.

Q: Why build your own MinHash implementation instead of using `datasketch`? A: `datasketch` transitively

imports `scipy.integrate.quad`, which can trigger a hang-prone subprocess spawn on Windows under certain security software - a real, reproduced compatibility bug, not a style preference.

Q: Why is community-aware retrieval opt-in but weighted retrieval is default-on? A: Weighted retrieval is a strict quality improvement with no failure mode worse than the old behavior. Community-aware retrieval can, if misapplied, wrongly exclude relevant results - its confidence-gated fallback mitigates that, but the team chose to keep it opt-in until more confidence is built up in production use.

Q: Why does pre-execution validation never block on UNKNOWN? A: Because "the graph doesn't mention this" is the normal case for a large fraction of real, correct code (anything not yet extracted, or genuinely new code) - blocking on it would make the tool unusably noisy. Blocking is reserved for cases the graph can *prove* wrong.

"What happens if X fails?" Questions

Q: What if the LLM backend isn't configured when `reason` is called? A: Retrieval still runs successfully; the function returns the retrieved graph context along with an explicit message that reasoning can't proceed without a provider - it doesn't crash or silently return nothing.

Q: What if a proposed code snippet is incomplete/unparseable for preexec validation? A: `ast.parse` fails gracefully; the validator falls back to regex-based reference extraction so it still produces a best-effort check rather than erroring out.

Q: What if the graph file is corrupted or missing when the MCP server starts a query? A: `_load_graph` catches `JSONDecodeError` and returns a friendly message suggesting a rebuild, rather than raising a stack trace to the client.

Q: What if community-aware retrieval's seed nodes span two unrelated modules? A: It's specifically designed to detect this (`seeds_outside/no_signal` fallback reasons) and falls back to searching the full graph rather than guessing which module is "right."

Contribution Questions

Q: Which of your four backend features required the least new infrastructure, and why? A: Semantic graph - it reuses the existing multi-backend `llm._call_llm()` function unchanged, so no new LLM plumbing was needed, only new prompt construction and a new merge strategy.

Q: Which feature is riskiest to ship enabled by default, and why is it not default-on? A: Community-aware retrieval - a wrong community restriction could hide the right answer. It's opt-in, with an automatic full-graph fallback, until more real-world confidence is built.

Q: How did you verify the reasoning feature's self-correction actually works, rather than just trusting the design? A: A dedicated test scripts an LLM provider to first answer wrong, checks that the contradiction is detected, and then asserts the second (LLM-corrected) answer is what's actually returned to the caller.

Difficult / Tricky Questions

Q: Doesn't community-aware retrieval essentially do the same job as the original clustering in `cluster.py`? Why two systems? A: They serve different phases. `cluster.py` runs once at build time to label the saved graph. `community_detection.py` runs at query time, needs to handle degenerate cases (empty/edgeless/tiny graphs) that don't matter at build time, and is designed as a pluggable interface with algorithms (like label propagation) the build-time clusterer doesn't have. `community_index.py` explicitly prefers the build-

time labels when they exist, only falling back to live detection - so the two systems cooperate rather than duplicate effort.

Q: Your retrieval-only accuracy metric (precision@10) shows plain grep beating your graph approach. Isn't that a sign the whole approach doesn't work?

A: It shows the *narrow* metric isn't graph retrieval's strength - graphify is designed to hand an LLM broad supporting context, not act as a precision-ranked search engine by itself. The metric that actually reflects real-world usefulness - does the retrieved context help an LLM produce correct code - shows the opposite result, with our changes closing most of the gap to the original engine and both graph approaches beating no-context/BM25 substantially. We report both numbers rather than only the flattering one, because that's the honest picture.

Q: The "clean Graphify baseline codebase" commit touches almost every file with huge diffs - how do you know that's not where you actually rewrote the engine, undermining your claim about what's "original"?

A: We checked directly, not assumed - a diff of that commit shows it strips explanatory comments and reformats code (e.g., collapsing/expanding dict literals), with the underlying logic unchanged. We verified this on `build.py` specifically by reading the diff line by line.

Q: If community-aware retrieval and weighted retrieval are independent flags, what happens if they conflict? A: They don't operate on the same axis, so there's no real conflict: community-aware decides *which subgraph* to search; weighted decides *how* to rank/traverse within whatever subgraph is chosen.

`community_aware_expand()` explicitly composes the two by restricting the graph first, then calling the same weighted `expand_neighborhood()` on the result.

Q: Why does the preexec validation hook only support Python well? A: Python has a reliable standard-library AST module that makes real static analysis of a code snippet cheap and dependency-free. Supporting another language equally well would mean either shipping that language's own parser or building a second tree-sitter-based extractor specifically for validation - the team scoped this explicitly as a known limitation rather than shipping a false sense of coverage.

21. Learning Path: New to the Project

Read in this order if you've never opened this repository:

1. **What is Graphify?** - read Section 2 above. It parses a codebase into a graph of typed relationships.
2. **What problem does it solve?** - raw files are expensive/imprecise for an LLM; keyword search is structurally blind; a graph preserves relationships.
3. **What does our project change?** - four backend features (weighted retrieval, community-aware retrieval, semantic graph, bidirectional reasoning + preexec validation) plus a full VS Code extension.
4. **What are the major components?** - the Python `graphify/` engine (extraction, graph building, retrieval), and the TypeScript `vscode-extension/` client.
5. **How do the components communicate?** - the extension shells out to the Python CLI as a subprocess; there is no server, no shared memory, no MCP between them (MCP is a *separate*, optional interface for other AI clients).
6. **How does a request flow?** - see Section 10's two traced examples (Copilot Chat query, and the `reason` command).
7. **What are our contributions, concretely?** - see Section 5's table; each has its own file(s) and its own research doc under `docs/research/`.
8. **How is everything implemented?** - read Section 7 (backend features) and Section 8 (extension) for the real mechanics, not just names.
9. **What did we actually measure?** - Section 17; know both the flattering number (CrossCodeEval) and the honest caveat (precision@10).
10. **What's still uncertain or unfinished?** - Section 18 and Section 23; know the difference between "not implemented yet" and "implemented but limited."

The 10 things that matter most - if you understand these, you understand the project:

1. Graphify = codebase -> knowledge graph (nodes = code entities, edges = typed/confidence-tagged relationships).
2. The "seven-stage pipeline" is a conceptual description; the real code is ~1000 imperative lines in `cli.py`'s `extract` command.
3. Weighted retrieval ranks relations by importance and uses that for both Dijkstra shortest-path and priority-ordered neighborhood expansion.
4. Community-aware retrieval restricts search to a confidence-scored cluster, with automatic full-graph fallback.
5. Semantic graph adds a closed vocabulary of LLM-inferred intent edges, never overwriting existing structural edges.
6. Bidirectional reasoning makes an LLM state checkable claims, verifies them against the graph, and forces a self-correction on contradiction.
7. Pre-execution validation blocks only provably-wrong AI-agent code proposals - never merely unknown ones.
8. The VS Code extension has no server - it's a thin CLI-shelling client, plus two Copilot Chat tools.
9. Our measured result: retrieval improvements raised real code-completion exact-match accuracy by roughly 10 points over the pre-existing engine, while honestly reporting that a narrow precision@10 metric still favors plain grep.

10. Everything opt-in (community-aware, semantic-graph, reasoning, preexec) fails safe: falls back, refuses to overclaim, or simply doesn't block - never silently wrong with high confidence.

22. Cheat Sheet

Project in 30 seconds

Synapse turns a codebase into a knowledge graph and layers smarter retrieval, an LLM-verification loop, and a VS Code + Copilot integration on top of the open-source Graphify engine - measurably improving real code-completion accuracy over the original tool.

Project in 1 minute

Graphify parses code (and docs/papers/images) into a graph of typed, confidence-tagged relationships, so questions can be answered by walking a graph instead of reading raw files. We forked it and added: weighted retrieval (prefer strong relationships over weak ones), community-aware retrieval (search the relevant cluster first, with safe fallback), a semantic-graph pass (LLM-labeled intent edges), a bidirectional-reasoning loop (LLM claims verified against the graph, self-corrected on contradiction), a pre-execution validation hook (blocks provably-wrong AI-agent code writes), and a full VS Code extension with two GitHub Copilot Chat tools. We measured the effect with a real-LLM benchmark: exact-match code-completion accuracy rose from 21.1% (original engine) to 31.6% (ours) on a paired sample.

Project in 3 minutes

Use Section 1 (Executive Summary) verbatim, then add one sentence each from Section 17 (both accuracy numbers, including the honest grep caveat) and Section 18 (top 2 limitations).

Original Graphify

Python CLI, tree-sitter-based extraction across 25+ languages, NetworkX graph, Leiden/Louvain community detection, multi-backend LLM semantic extraction for docs/papers/images, MCP server, CLI commands for query/path/explain/export, strong security hardening (SSRF guards, path traversal guards, prompt-injection defenses).

Our contributions

Weighted retrieval - Community-aware retrieval - Semantic graph - Bidirectional reasoning + pre-execution validation - Synapse VS Code extension (Dashboard, sidebar, graph viz, 2 Copilot tools, onboarding) - Two accuracy-evaluation harnesses.

Architecture

VS Code UI -> `extension.ts/lmTools.ts` -> `cliService.ts` (`child_process.spawn`) -> Python `graphify` CLI -> original pipeline + our retrieval/verification layers -> optional multi-provider LLM backend.

Technologies

Python, tree-sitter, NetworkX, Leiden algorithm (graspologic), rapidfuzz, a custom MinHash/LSH implementation, multi-provider LLM SDKs, MCP, TypeScript, VS Code Extension API + Language Model Tools API, vis.js, pytest, uv.

Most important algorithms

Tree-sitter AST parsing - MinHash+LSH dedup blocking - Jaro-Winkler verification - Leiden/Louvain community detection - Dijkstra shortest path (weighted retrieval) - priority-ordered BFS/DFS (weighted neighborhood expansion) - betweenness centrality (surprise detection) - cycle detection (import cycles).

Most important files

```
graphify/extract.py, graphify/build.py, graphify/cluster.py, graphify/analyze.py, graphify/weights.py, graphify/weighted_retrieval.py, graphify/community_retrieval.py, graphify/semantic_graph.py, graphify/graph_checks.py, graphify/bidirectional_reasoner.py, graphify/preexec_validate.py, graphify/serve.py, vscode-extension/src/cliService.ts, vscode-extension/src/lmTools.ts.
```

Most important functions

```
extract.extract(), build.build_from_json(), cluster.cluster(), weighted_retrieval.expand_neighborhood() / find_shortest_path(), community_retrieval.community_aware_expand(), semantic_graph.apply_semantic_relations(), graph_checks.check_relation(), bidirectional_reasoner.reason(), preexec_validate.validate_proposal(), cliService.runGraphify().
```

20 must-know viva questions

1. What is Graphify, in one sentence?
2. What problem does a knowledge graph solve that keyword search doesn't?
3. What are the three confidence tags and what do they mean?
4. Where exactly is the fork boundary between upstream Graphify and our code?
5. Name our four backend contributions and one sentence on each.
6. Is weighted retrieval's neighborhood expansion literally Dijkstra? (No - only shortest-path is; expansion is priority-ordered BFS/DFS.)
7. How does community-aware retrieval decide when NOT to restrict the search?
8. How does semantic-graph avoid overwriting existing structural edges?
9. What does "bidirectional" mean in bidirectional reasoning?
10. What's the difference between UNKNOWN and CONTRADICTED, and why does it matter?
11. Why does pre-execution validation never block on UNKNOWN?
12. Does the VS Code extension run a server? (No - it shells out to the CLI.)
13. What are the two GitHub Copilot Chat tools called, and what does each take as input?
14. Is `graphify serve` a CLI subcommand? (No - separate entry point `graphify-mcp`.)
15. What LLM providers are supported?
16. How is prompt injection defended against?
17. What were the two accuracy numbers you measured, and which is more important?
18. Why does grep still win on precision@10, and why doesn't that undermine the project?
19. Name one thing that is NOT yet implemented (future work), and one thing that IS implemented but limited.
20. What's the one thing you'd improve next if you had more time?

23. Things the Team Must Verify

Flagged explicitly rather than guessed, per the accuracy requirement of this document:

1. **Whether `cli.py`'s affected command is original Graphify or a team addition** - its style doesn't obviously mark it either way, and it wasn't in scope for the deep pipeline trace. Confirm before claiming it either way in a presentation.
 2. **Whether the MCP tools `list_prs/get_pr_impact/triage_prs` (backed by `graphify/prs.py`) are original or a separate team contribution** - not traced in this pass; their tool descriptions don't self-identify.
 3. **Push the local merge of `origin/main` into `main`** performed during this analysis (commit `81bdc08`, bringing in the reasoning/preexec-validation feature) so GitHub and the local checkout agree. Currently local `main` is 4 commits ahead of `origin/main` and unpushed.
 4. **Confirm the 7 new `test_security.py` failures are a network-sandbox artifact, not a regression.** A fresh full run during this analysis (post-merge of all four backend features) produced 3,743 passed / 49 failed / 46 skipped - 7 more failures than the previously-documented 42, all in `test_security.py`'s DNS-dependent SSRF-guard tests, in an environment that very likely has no outbound network access. Re-run `uv run pytest tests/ -q` on a normal machine with network access and, ideally, also re-run against an isolated pre-merge worktree (as was done for the earlier 3-feature merge) to get a clean, final regression count for the reasoning/preexec-validation merge specifically.
 5. **The PyPI package name `graphify`** (vs. the `graphify` import/CLI name) - confirm with the team whether this is intentional (e.g., name-squatting avoidance) or an unresolved placeholder, since it's an easy point of confusion for a panel to seize on.
 6. **Refresh the self-graph** at `graphify-out/` (built from commit `d0c8c6f`, before any of the four backend features existed) - run `graphify update` . if the team wants to use Synapse's own self-analysis tooling on the current codebase going forward.
 7. **Confirm whether the Electron-based VS Code integration test suite** (4 tests, requires a downloaded VS Code test binary) currently passes - it was not re-run in the most recent evaluation session per `docs/eval/SYNAPSE_VS_GRAPHIFY.md`.
 8. **A full `n=47` CrossCodeEval run for the "Existing Graphify" comparison arm** is recommended once API quota allows, to replace the current `n=19` partial run with a complete one.
-

24. Appendix - File & Function Index

A quick-lookup table of the files most likely to come up in questions, and what to say about each.

File	One-line description
<code>graphify/detect.py</code>	Discovers and classifies files; filters secrets; incremental change detection
<code>graphify/extract.py</code>	Extraction dispatch + cross-file resolution (imports, inferred calls, ID canonicalization)
<code>graphify/extractors/engine.py</code>	Shared tree-sitter engine for ~15 languages
<code>graphify/extractors/rust.py</code>	Self-contained, hand-written example extractor (good for a live walkthrough)
<code>graphify/build.py</code>	Assembles the NetworkX graph; dedup; cross-language edge guard
<code>graphify/dedup.py</code> , <code>_minhash.py</code>	Fuzzy entity dedup (MinHash/LSH + Jaro-Winkler) for non-code nodes
<code>graphify/cluster.py</code>	Build-time Leiden/Louvain community detection
<code>graphify/analyze.py</code>	God nodes, surprises, suggested questions, import cycles
<code>graphify/report.py</code>	Renders <code>GRAPH_REPORT.md</code> (function: <code>generate()</code>)
<code>graphify/export.py</code> , <code>exporters/</code>	All output formats
<code>graphify/cache.py</code>	SHA-256 content-hash caching
<code>graphify/security.py</code>	SSRF/path-traversal/label-sanitization guards
<code>graphify/llm.py</code>	Multi-backend LLM client
<code>graphify/cli.py</code>	Command dispatch; home of the <code>extract</code> pipeline orchestration
<code>graphify/serve.py</code>	MCP server (separate entry point <code>graphify-mcp</code>)
<code>graphify/weights.py</code>	<code>RelationWeightRegistry</code> - our importance table
<code>graphify/weighted_retrieval.py</code>	Our weighted BFS/DFS + Dijkstra shortest path
<code>graphify/community_detection.py</code> , <code>community_index.py</code> , <code>community_retrieval.py</code>	Our query-time community-aware retrieval
<code>graphify/semantic_extraction.py</code> , <code>semantic_relations.py</code> , <code>semantic_graph.py</code>	Our LLM-inferred intent-edge pass
<code>graphify/graph_checks.py</code> , <code>claims.py</code> , <code>bidirectional_reasoner.py</code> , <code>preexec_validate.py</code>	Our claim-verification + code-write safety features
<code>vscode-extension/src/extension.ts</code>	Extension activation + command registry
<code>vscode-extension/src/cliService.ts</code>	The single choke point for all Python CLI calls

<code>vscode-extension/src/lmTools.ts</code>	The two GitHub Copilot Chat tools
<code>tools/eval/retrieval_accuracy.py, crosscodeeval.py</code>	Our two accuracy-measurement harnesses
<code>docs/eval/ACCURACY_REPORT.md, SYNAPSE_VS_GRAPHIFY.md</code>	Our written accuracy reports
<code>docs/research/*.md</code>	Our own design-rationale docs, one per backend feature

This document was produced by tracing the actual source code, git history, and test suite of this repository. Every architectural claim, algorithm description, and number in this document is anchored to something read directly from the repository during this analysis - not inferred from filenames or general knowledge of similar tools. Section 23 lists the small number of points that could not be fully confirmed and should be checked with the team before presenting them as fact.